

Prova 1 - Inteligencia Artificial

Template de Resolucao

Aluno(a): _____

Maio de 2026

Contents

1	Questao 1 - BFS, DFS e Busca Iterativa em Profundidade	3
1.1	Formulacao do problema em espaco de estados	3
1.2	Grafo definido no enunciado	4
1.3	Arvore de busca da BFS (com ordem de expansao)	5
1.4	Fronteira e visitados da BFS	5
1.5	Resultado final da BFS	6
1.6	Arvore de busca da DFS (com ordem de expansao)	6
1.7	Fronteira e visitados da DFS	6
1.8	Resultado final da DFS	7
1.9	Busca iterativa em profundidade (IDS)	7
1.10	Arvores da IDS em todas as iteracoes (limites 0 a 4)	8
1.11	Fronteira e visitados da IDS (iteracao limite 4)	8
1.12	Resultado final da IDS	9
1.13	Experimento adicional: alteracao da ordem de sucessores	9
1.14	Redesenho completo apos a alteracao de sucessores	10
1.15	Respostas objetivas do experimento	12
1.16	Comparacao final entre os algoritmos	13
2	Questao 2 - Greedy Best-First Search e A*	14
2.1	Formulacao do problema em espaco de estados	14
2.2	Dados do grafo e heuristica	15
2.3	Greedy Best-First Search ($f(n) = h(n)$)	16
2.4	A* ($f(n) = g(n) + h(n)$)	17
2.5	Experimento: alterar heuristica de exatamente 3 nos	19
2.6	Respostas objetivas do experimento	20
2.7	Comparacao final: Greedy vs A*	21
3	Questao 3 - Busca Local (8-Rainhas)	21
3.1	Formulacao em espaco de estados	21
3.2	Hill-Climbing (execucao manual)	22
3.3	Top-5 vizinhos por iteracao (HC)	22
3.4	Todos os vizinhos avaliados por iteracao (HC)	23
3.5	Analise de armadilha de busca	29
3.6	Random Restart Hill-Climbing (20 execucoes)	29

3.7	Simulated Annealing	30
3.8	Comparacao final (Q3)	31
4	Questao 4 - CSP e Otimizacoes de Busca	31
4.1	Modelagem do CSP	31
4.2	Grafo de restricoes	32
4.3	Backtracking (manual + implementado)	32
4.4	MRV + Degree	32
4.5	Forward Checking	32
4.6	Backjumping	33
4.7	Comparacao quantitativa (Q4)	33
4.8	Analise final pedida na questao	33
5	Questao 5 - Minimax e Poda Alpha-Beta	34
5.1	Conceitos pedidos no enunciado	34
5.2	Dados da arvore	34
5.3	Minimax passo a passo	34
5.4	Arvore parcialmente preenchida (visao estrutural)	35
5.5	Alpha-Beta (mesma ordem do enunciado)	35
5.6	Representacao visual da poda Alpha-Beta	36
5.7	Reordenacao para maximizar podas	36
5.8	Minimax limitado (profundidade 2)	36
5.9	Experimento adicional: alterar 3 folhas	37
5.10	Desenhos pos-experimento (Minimax e Alpha-Beta)	37
5.11	Comparacao final (Q5)	38
6	Questao 6 - Monte Carlo Tree Search (MCTS)	38
6.1	Escopo e modelagem usada	38
6.2	Explicacao do MCTS (itens teoricos do enunciado)	39
6.3	Formula UCT usada	39
6.4	Execucao detalhada (10 iteracoes)	39
6.5	Arvore parcial apos as 10 iteracoes	40
6.6	Tabela UCT apos 10 iteracoes	40
6.7	Sensibilidade ao parametro C (rollout semi-guloso)	40
6.8	Rollout aleatorio vs semi-guloso (C=1.4)	41
6.9	Efeito do numero de iteracoes	41
6.10	Comparacao final pedida na Q6	42
6.11	Resposta objetiva da jogada sugerida	42
A	Codigos Python	42
A.1	Questao 1	42
A.2	Questao 2	48
A.3	Questao 3	54
A.4	Questao 4	63
A.5	Questao 5	73
A.6	Questao 6	79

Orientacoes Rapidas

- Use o PDF da prova como referencia oficial para grafos/figuras.
- Preencha cada secao ‘qX.tex’ com modelagem, tabelas, analise e conclusoes.
- Para arvores, use os modelos com ‘forest’/‘tikz’ e ajuste os nos.
- Rode os codigos Python localmente e transcreva os resultados para as tabelas.

1 Questao 1 - BFS, DFS e Busca Iterativa em Profundidade

1.1 Formulacao do problema em espaco de estados

a) Definicao formal do estado

$$P = \langle Z, \mathcal{A}, T, s_0, G, c \rangle$$

- Z : conjunto de estados (salas do predio), com

$$Z = \{A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S\}.$$

- Cada estado $z \in Z$ representa a sala atual do agente.
- $\mathcal{A}$: conjunto de acoes de deslocamento entre salas adjacentes.
- T : funcao de transicao que leva da sala atual para uma sala vizinha valida.

b) Estado inicial

$$s_0 = A$$

c) Teste de objetivo

$$G(z) = \begin{cases} \text{verdadeiro,} & \text{se } z = S \\ \text{falso,} & \text{caso contrario} \end{cases}$$

d) Funcao sucessora

- A funcao sucessora devolve os vizinhos da sala atual respeitando a ordem obrigatoria do enunciado.
- Exemplo: $\text{Suc}(A) = [B, C, D]$, $\text{Suc}(C) = [G, H]$, $\text{Suc}(F) = [K, L]$.

e) Funcao de custo

$$c(z, a, z') = 1$$

Cada movimento entre salas adjacentes tem custo unitario. Logo, o custo total de um caminho e o numero de arestas percorridas.

f) Representacao escolhida em Python

```

1 grafo = {
2     "A": ["B", "C", "D"],
3     "B": ["E", "F"],
4     "C": ["G", "H"],
5     "D": ["I"],
6     "E": ["J"],
7     "F": ["K", "L"],
8     "G": ["M"],
9     "H": ["N", "O"],
10    "I": ["P"],
11    "J": [],
12    "K": ["Q"],
13    "L": [],
14    "M": ["R"],
15    "N": [],
16    "O": ["S"],
17    "P": [],
18    "Q": [],
19    "R": [],
20    "S": [],
21 }
22 estado_inicial = "A"
23 estado_objetivo = "S"

```

1.2 Grafo definido no enunciado

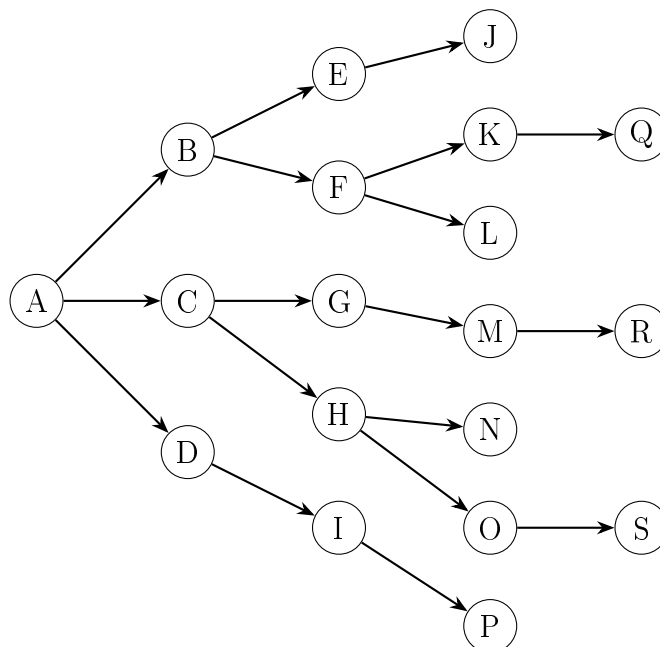


Figure 1: Grafo da Q1

1.3 Arvore de busca da BFS (com ordem de expansao)

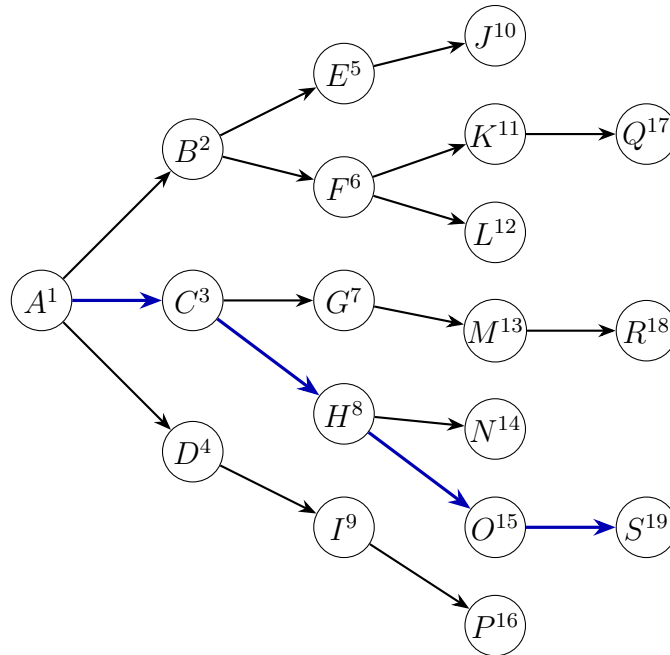


Figure 2: Arvore de busca BFS com indice de expansao; caminho solucao destacado em azul

1.4 Fronteira e visitados da BFS

Convencao usada:

- *Fronteira* = fila de abertos (da esquerda para a direita).
- *Visitados* = ordem de nos efetivamente expandidos.
- Estado inicial: *fronteira* = [A] e *visitados* = $\emptyset$.

Passo	No expandido	Fronteira (abertos)	Visitados
0	-	[A]	[]
1	A	[B, C, D]	[A]
2	B	[C, D, E, F]	[A, B]
3	C	[D, E, F, G, H]	[A, B, C]
4	D	[E, F, G, H, I]	[A, B, C, D]
5	E	[F, G, H, I, J]	[A, B, C, D, E]
6	F	[G, H, I, J, K, L]	[A, B, C, D, E, F]
7	G	[H, I, J, K, L, M]	[A, B, C, D, E, F, G]
8	H	[I, J, K, L, M, N, O]	[A, B, C, D, E, F, G, H]
9	I	[J, K, L, M, N, O, P]	[A, B, C, D, E, F, G, H, I]
10	J	[K, L, M, N, O, P]	[A, B, C, D, E, F, G, H, I, J]
11	K	[L, M, N, O, P, Q]	[A, B, C, D, E, F, G, H, I, J, K]
12	L	[M, N, O, P, Q]	[A, B, C, D, E, F, G, H, I, J, K, L]
13	M	[N, O, P, Q, R]	[A, B, C, D, E, F, G, H, I, J, K, L, M]

14	N	[O, P, Q, R]	[A, B, C, D, E, F, G, H, I, J, K, L, M, N]
15	O	[P, Q, R, S]	[A, B, C, D, E, F, G, H, I, J, K, L, M, N, O]
16	P	[Q, R, S]	[A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P]
17	Q	[R, S]	[A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q]
18	R	[S]	[A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R]
19	S	[]	[A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S]

1.5 Resultado final da BFS

- Caminho solucao: $A \rightarrow C \rightarrow H \rightarrow O \rightarrow S$.
- Profundidade da solucao: 4.
- Custo da solucao: 4.
- Nos gerados: 19.
- Nos expandidos: 19.

1.6 Arvore de busca da DFS (com ordem de expansao)

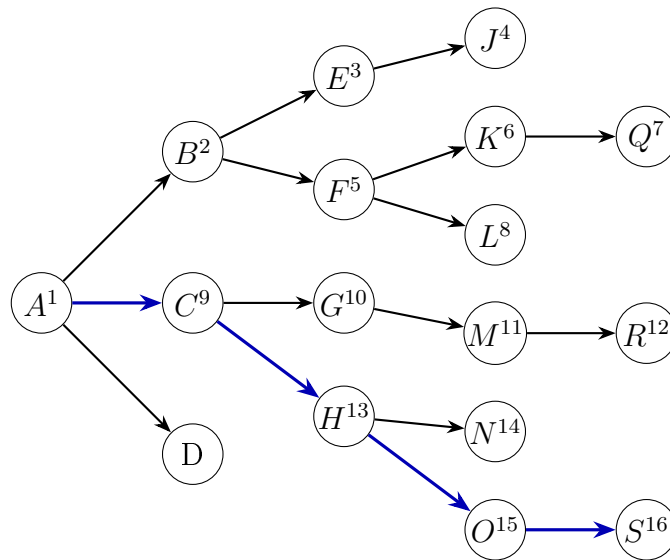


Figure 3: Arvore de busca DFS com indice de expansao; caminho solucao em azul

1.7 Fronteira e visitados da DFS

- Fronteira = pilha de abertos.
- O topo da pilha esta na extrema direita da lista.
- Estado inicial: $\text{fronteira} = [A]$ e $\text{visitados} = \emptyset$.

Passo	No expandido	Fronteira (pilha)	Visitados
0	-	[A]	[]
1	A	[D, C, B]	[A]
2	B	[D, C, F, E]	[A, B]
3	E	[D, C, F, J]	[A, B, E]
4	J	[D, C, F]	[A, B, E, J]
5	F	[D, C, L, K]	[A, B, E, J, F]
6	K	[D, C, L, Q]	[A, B, E, J, F, K]
7	Q	[D, C, L]	[A, B, E, J, F, K, Q]
8	L	[D, C]	[A, B, E, J, F, K, Q, L]
9	C	[D, H, G]	[A, B, E, J, F, K, Q, L, C]
10	G	[D, H, M]	[A, B, E, J, F, K, Q, L, C, G]
11	M	[D, H, R]	[A, B, E, J, F, K, Q, L, C, G, M]
12	R	[D, H]	[A, B, E, J, F, K, Q, L, C, G, M, R]
13	H	[D, O, N]	[A, B, E, J, F, K, Q, L, C, G, M, R, H]
14	N	[D, O]	[A, B, E, J, F, K, Q, L, C, G, M, R, H, N]
15	O	[D, S]	[A, B, E, J, F, K, Q, L, C, G, M, R, H, N, O]
16	S	[D]	[A, B, E, J, F, K, Q, L, C, G, M, R, H, N, O, S]

1.8 Resultado final da DFS

- Caminho solucao: $A \rightarrow C \rightarrow H \rightarrow O \rightarrow S$.
- Profundidade da solucao: 4.
- Custo da solucao: 4.
- Nos gerados: 17.
- Nos expandidos: 16.

1.9 Busca iterativa em profundidade (IDS)

Resumo por limite de profundidade:

Limite	Objetivo encontrado?	Nos gerados	Nos expandidos
0	nao	1	1
1	nao	4	4
2	nao	9	9
3	nao	16	16
4	sim	17	16

Table 3: IDS por iteracao de limite

Ordem de visita aos nos (IDS):

Limite	Ordem de visita (expansao)
0	[A]
1	[A, B, C, D]
2	[A, B, E, F, C, G, H, D, I]
3	[A, B, E, J, F, K, L, C, G, M, H, N, O, D, I, P]
4	[A, B, E, J, F, K, Q, L, C, G, M, R, H, N, O, S]

Table 4: IDS: ordem de visita dos nos em cada iteracao

1.10 Arvores da IDS em todas as iteracoes (limites 0 a 4)

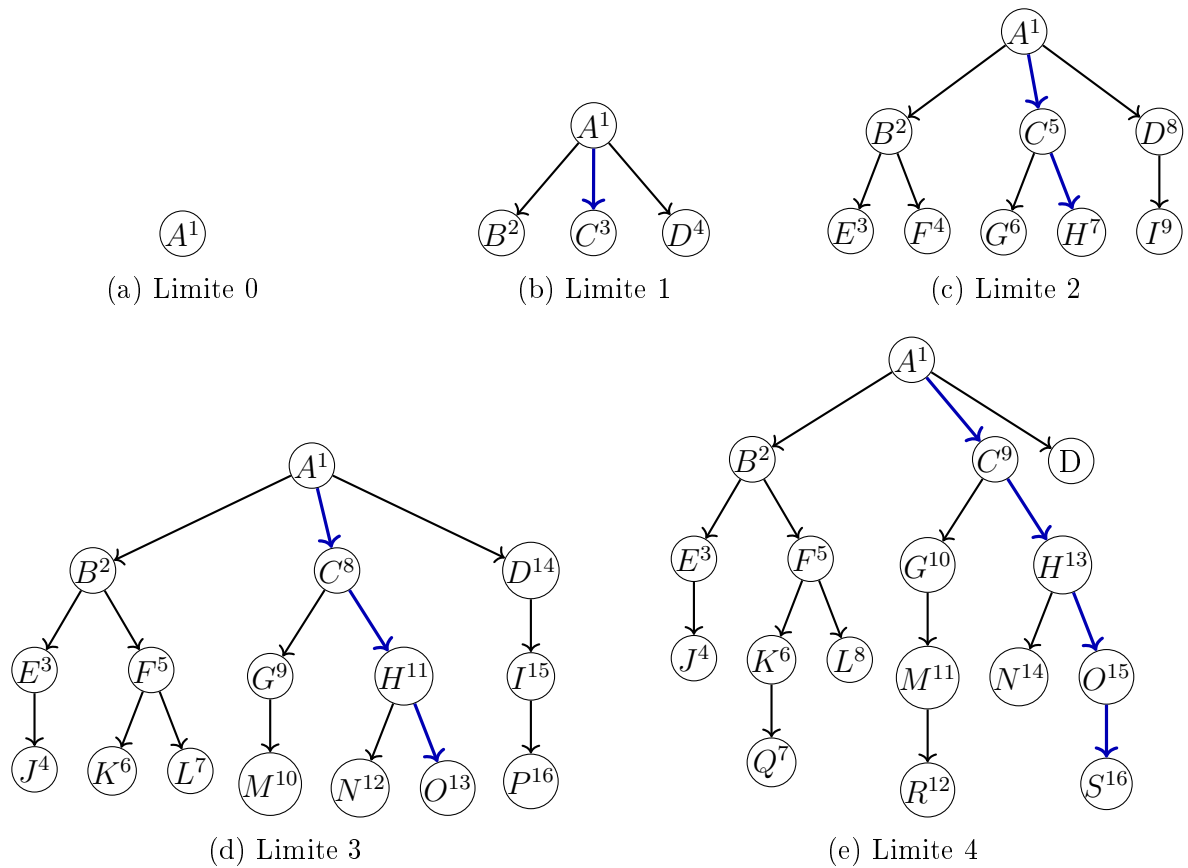


Figure 4: IDS: uma arvore por iteracao de limite; caminho solucao em azul

1.11 Fronteira e visitados da IDS (iteracao limite 4)

- Nesta tabela, mostramos a iteracao que encontrou o objetivo.
- A fronteira e tratada como pilha (topo na extrema direita).

Passo	No expandido	Fronteira (pilha)	Visitados
0	-	[A]	[]
1	A	[D, C, B]	[A]
2	B	[D, C, F, E]	[A, B]

3	E	[D, C, F, J]	[A, B, E]
4	J	[D, C, F]	[A, B, E, J]
5	F	[D, C, L, K]	[A, B, E, J, F]
6	K	[D, C, L, Q]	[A, B, E, J, F, K]
7	Q	[D, C, L]	[A, B, E, J, F, K, Q]
8	L	[D, C]	[A, B, E, J, F, K, Q, L]
9	C	[D, H, G]	[A, B, E, J, F, K, Q, L, C]
10	G	[D, H, M]	[A, B, E, J, F, K, Q, L, C, G]
11	M	[D, H, R]	[A, B, E, J, F, K, Q, L, C, G, M]
12	R	[D, H]	[A, B, E, J, F, K, Q, L, C, G, M, R]
13	H	[D, O, N]	[A, B, E, J, F, K, Q, L, C, G, M, R, H]
14	N	[D, O]	[A, B, E, J, F, K, Q, L, C, G, M, R, H, N]
15	O	[D, S]	[A, B, E, J, F, K, Q, L, C, G, M, R, H, N, O]
16	S	[D]	[A, B, E, J, F, K, Q, L, C, G, M, R, H, N, O, S]

1.12 Resultado final da IDS

- Caminho solucao: $A \rightarrow C \rightarrow H \rightarrow O \rightarrow S$.
- Profundidade da solucao: 4.
- Custo da solucao: 4.
- Nos gerados totais (somando limites 0..4): 47.
- Nos expandidos totais (somando limites 0..4): 46.

1.13 Experimento adicional: alteracao da ordem de sucessores

Alteracoes aplicadas (exatamente dois nos):

- $C : [G, H] \rightarrow [H, G]$
- $F : [K, L] \rightarrow [L, K]$

Resultados apos reexecucao dos tres algoritmos:

Algoritmo	Caminho	Prof.	Custo	Gerados	Expandidos
BFS (original)	$A \rightarrow C \rightarrow H \rightarrow O \rightarrow S$	4	4	19	19
BFS (modificado)	$A \rightarrow C \rightarrow H \rightarrow O \rightarrow S$	4	4	19	18
DFS (original)	$A \rightarrow C \rightarrow H \rightarrow O \rightarrow S$	4	4	17	16
DFS (modificado)	$A \rightarrow C \rightarrow H \rightarrow O \rightarrow S$	4	4	15	13
IDS (original, total)	$A \rightarrow C \rightarrow H \rightarrow O \rightarrow S$	4	4	47	46
IDS (modificado, total)	$A \rightarrow C \rightarrow H \rightarrow O \rightarrow S$	4	4	45	43

Table 6: Comparacao numerica antes/depois da alteracao

1.14 Redesenho completo apos a alteracao de sucessores

Grafo apos alteracao em dois nos:

- $C : [H, G]$
- $F : [L, K]$

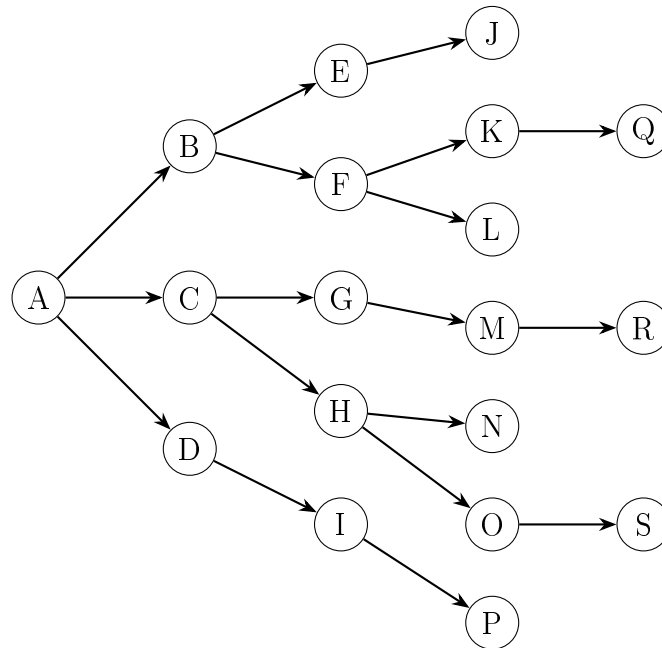


Figure 5: Grafo apos alteracao da ordem de sucessores (estrutura igual, ordem local alterada em C e F)

Arvore BFS (ordem modificada):

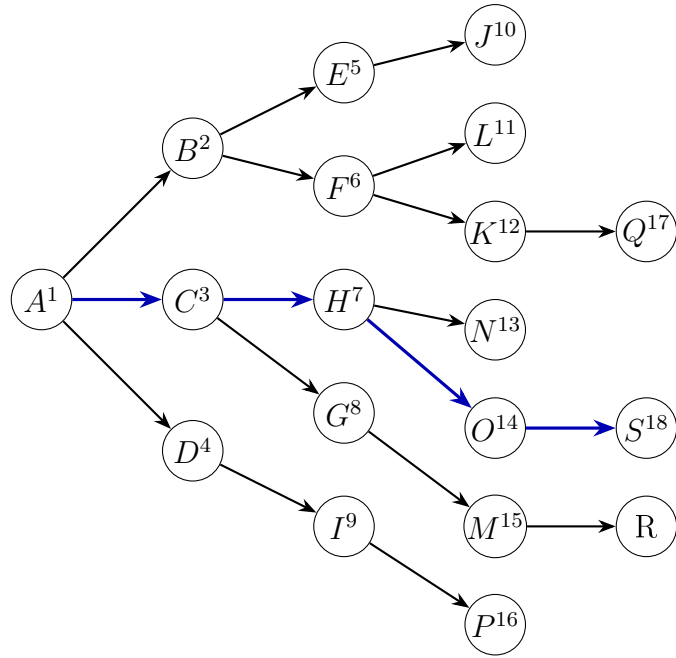


Figure 6: BFS apos alteracao; R permanece na fronteira quando S e encontrado

Arvore DFS (ordem modificada):

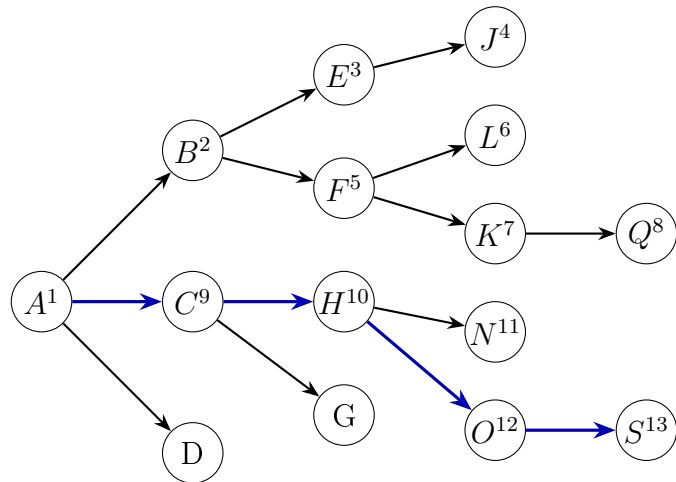


Figure 7: DFS apos alteracao; D e G permanecem na fronteira quando S e encontrado

Arvores IDS apos alteracao (limites 0 a 4):

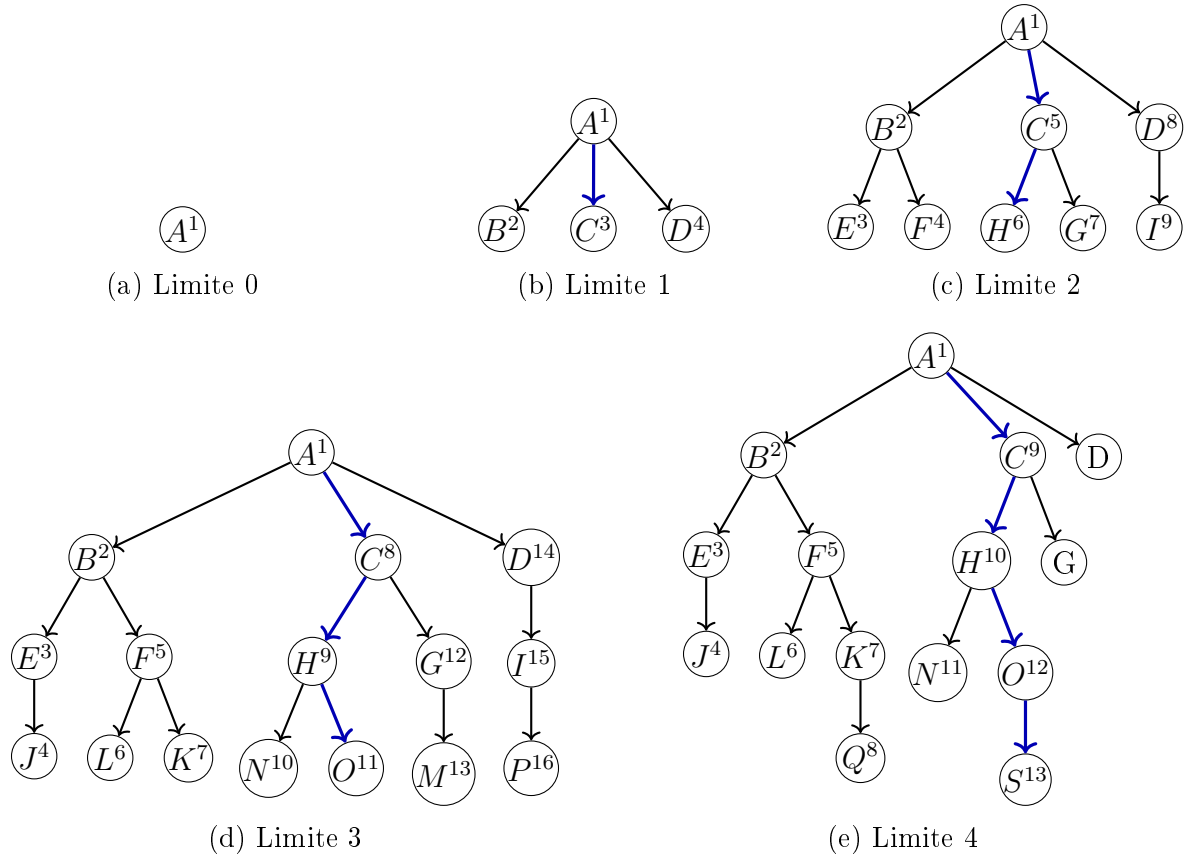


Figure 8: IDS apos alteracao: uma arvore por limite; caminho solucao em azul

Mudanca na ordem de expansao (resumo):

- BFS original: $[A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S]$.
- BFS modificado: $[A, B, C, D, E, F, H, G, I, J, L, K, N, O, M, P, Q, S]$.
- DFS original: $[A, B, E, J, F, K, Q, L, C, G, M, R, H, N, O, S]$.
- DFS modificado: $[A, B, E, J, F, L, K, Q, C, H, N, O, S]$.
- IDS (limite 4) original: $[A, B, E, J, F, K, Q, L, C, G, M, R, H, N, O, S]$.
- IDS (limite 4) modificado: $[A, B, E, J, F, L, K, Q, C, H, N, O, S]$.

1.15 Respostas objetivas do experimento

- a) O caminho solucao mudou? **Nao**. Em todos os casos permaneceu $A \rightarrow C \rightarrow H \rightarrow O \rightarrow S$.
- b) A quantidade de nos expandidos mudou? **Sim**. BFS: $19 \rightarrow 18$, DFS: $16 \rightarrow 13$, IDS (total): $46 \rightarrow 43$.
- c) Algum algoritmo foi mais sensivel a ordem dos sucessores? **Sim, DFS**. Foi o que mais reduziu expansoes (queda de 3, cerca de 18,75%).
- d) Qual algoritmo apresentou maior consumo de memoria? **BFS**. Maior fronteira maxima observada (7 nos), contra 4 em DFS/IDS.

- e) Qual algoritmo encontrou a solucao mais rapidamente? **DFS modificado**, com 13 nos expandidos.

1.16 Comparacao final entre os algoritmos

Notacao das complexidades:

- b : fator de ramificacao medio.
- d : profundidade da solucao mais rasa.
- m : profundidade maxima da arvore de busca.

a) Completeness

- BFS: completo em espaco finito e com b finito.
- DFS: nao completo em geral (pode prender em ramo muito profundo/infinito); neste problema finito, encontrou solucao.
- IDS: completo em espaco finito e com b finito.

b) Optimality

- BFS: otimo quando os custos de aresta sao unitarios (caso desta questao).
- DFS: nao e otimo em geral.
- IDS: otimo com custos unitarios (encontra a primeira solucao na menor profundidade).

c) Complexity of time

- BFS: $O(b^d)$.
- DFS: $O(b^m)$.
- IDS: $O(b^d)$ (com custo extra por reexpandir nos em limites menores).

d) Complexity of space

- BFS: $O(b^d)$ (guarda toda a fronteira do nivel).
- DFS: $O(bm)$ (guarda basicamente o caminho atual e alternativas locais).
- IDS: $O(bd)$ (parecido com DFS por iteracao).

e) Dependence on the order of successors

- BFS: media (a ordem afeta expansoes, mas menos que DFS).
- DFS: alta (a ordem muda fortemente o ramo seguido primeiro).
- IDS: media/alta (cada iteracao e DFS-limitada, entao herda essa sensibilidade).

f) Behavior in deep trees

- BFS: tende a estourar memoria.
- DFS: usa pouca memoria, mas pode descer em ramos ruins.

- IDS: boa alternativa quando ha limite de memoria, ao custo de repeticao.

g) **Adequacao para problemas grandes**

- BFS: limitado quando a fronteira cresce muito.
- DFS: bom quando memoria e critica e otimalidade nao e obrigatoria.
- IDS: bom compromisso entre completude e memoria em problemas de custo unitario.

2 Questao 2 - Greedy Best-First Search e A*

2.1 Formulacao do problema em espaco de estados

$$P = \langle Z, \mathcal{A}, T, z_0, G, c, h \rangle$$

- Z : conjunto de estados (nos do grafo), $Z = \{A, B, C, \dots, T\}$.
- $z_0 = A$: estado inicial.
- Objetivo: alcancar o estado T .
- $T(\cdot)$: funcao de transicao segundo as arestas dirigidas do enunciado.
- $c(z, a, z')$: custo da aresta percorrida.
- $h(z)$: heuristica fornecida no enunciado (estimativa ate T).

2.2 Dados do grafo e heurística

No	Sucessores (custo)	$h(n)$
A	B(2), C(4), D(3)	10
B	E(3), F(5)	8
C	G(4), H(6)	7
D	I(2)	9
E	J(4)	6
F	K(3), L(5)	5
G	M(6)	6
H	N(3), O(4)	4
I	P(5)	7
J	Q(4)	5
K	R(3)	3
L	-	6
M	S(2)	3
N	-	4
O	T(5)	1
P	-	8
Q	-	4
R	T(4)	2
S	T(3)	1
T	-	0

Table 7: Grafo da Q2 e valores heurísticos

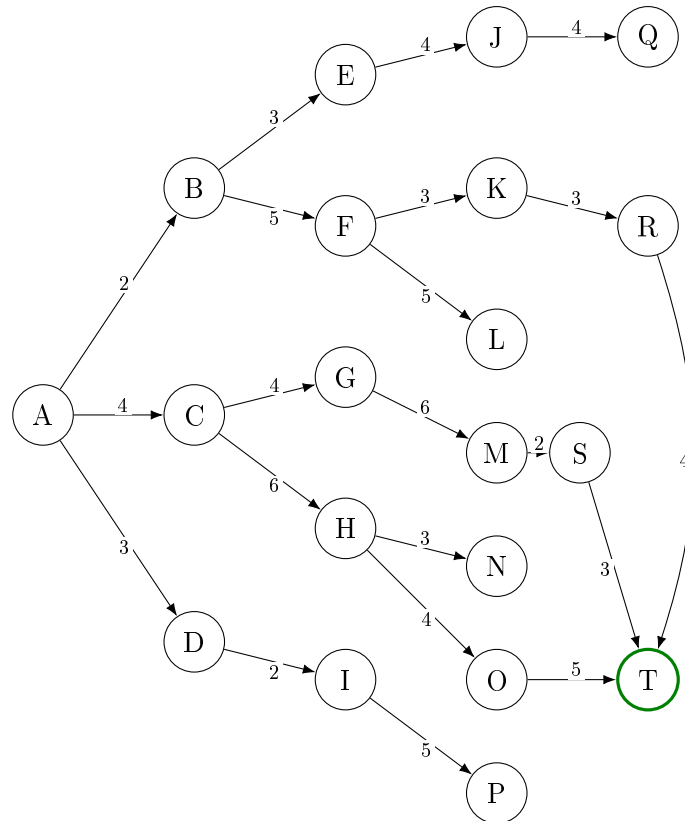


Figure 9: Representacao do grafo dirigido da Q2 (custos nas arestas)

Desempate adotado na implementacao:

- Menor prioridade primeiro (h no Greedy, $f = g + h$ no A^*).
- No Greedy, o custo acumulado $g(n)$ nao participa da prioridade.
- Em empate, ordem de insercao na fronteira.

2.3 Greedy Best-First Search ($f(n) = h(n)$)

Arvore de exploracao manual (Greedy):

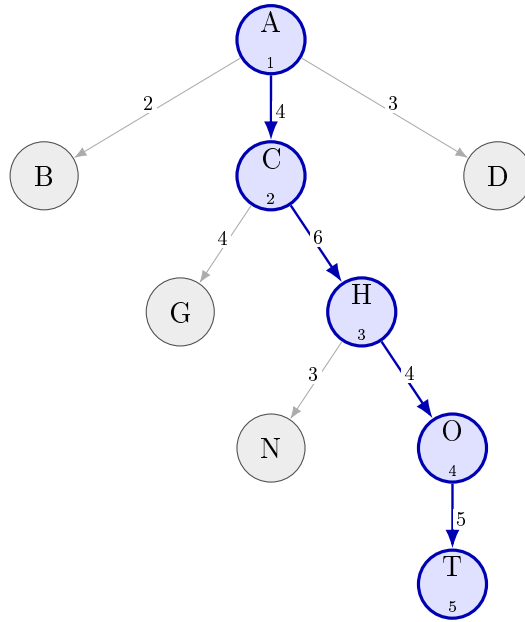


Figure 10: Arvore do Greedy ate encontrar T (azul = caminho solucao)

Passo	No	$h(n)$	Fronteira ordenada por h	Caminho parcial
1	A	10	[C(7), B(8), D(9)]	A
2	C	7	[H(4), G(6), B(8), D(9)]	A → C
3	H	4	[O(1), N(4), G(6), B(8), D(9)]	A → C → H
4	O	1	[T(0), N(4), G(6), B(8), D(9)]	A → C → H → O
5	T	0	[N(4), G(6), B(8), D(9)]	A → C → H → O → T

Resultado Greedy:

- Ordem de expansao: $[A, C, H, O, T]$.
- Caminho solucao: $A \rightarrow C \rightarrow H \rightarrow O \rightarrow T$.
- Profundidade: 4.
- Custo final: 19.
- Nos gerados: 9.
- Nos expandidos: 5.

2.4 A^* ($f(n) = g(n) + h(n)$)

Arvore de exploracao manual (A^*):

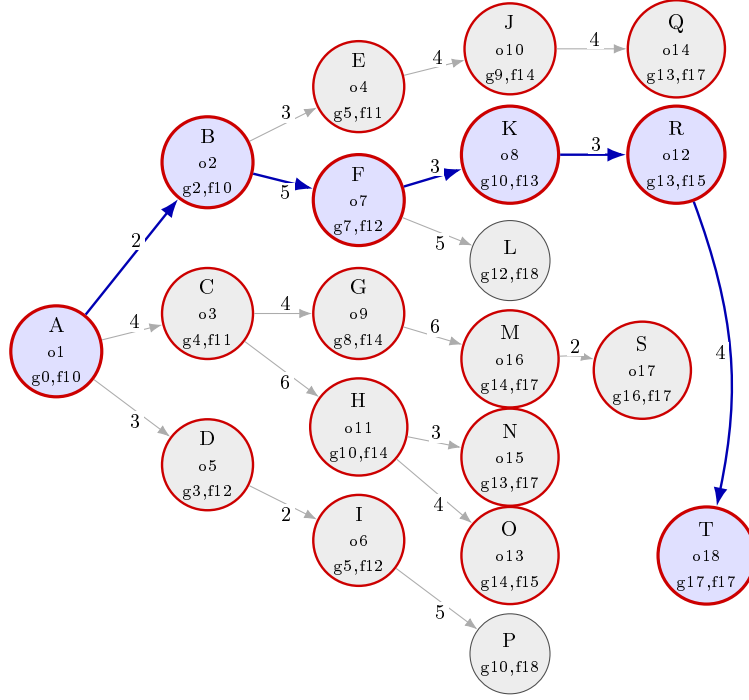


Figure 11: Arvore do A* (azul = caminho solucao; borda vermelha = no visitado; em cada no: ordem, g e f)

Passo	No	$g(n)$	$h(n)$	$f(n)$	Fronteira ordenada por f	Caminho parcial
1	A	0	10	10	[B(10), C(11), D(12)]	A
2	B	2	8	10	[C(11), E(11), D(12), F(12)]	A→B
3	C	4	7	11	[E(11), D(12), F(12), G(14), H(14)]	A→C
4	E	5	6	11	[D(12), F(12), G(14), J(14), H(14)]	A→B→E
5	D	3	9	12	[I(12), F(12), G(14), J(14), H(14)]	A→D
6	I	5	7	12	[F(12), G(14), J(14), H(14), P(18)]	A→D→I
7	F	7	5	12	[K(13), G(14), J(14), H(14), P(18), L(18)]	A→B→F
8	K	10	3	13	[G(14), J(14), H(14), R(15), P(18), L(18)]	A→B→F→K
9	G	8	6	14	[J(14), H(14), R(15), M(17), P(18), L(18)]	A→C→G
10	J	9	5	14	[H(14), R(15), Q(17), M(17), P(18), L(18)]	A→B→E→J
11	H	10	4	14	[R(15), O(15), N(17), Q(17), M(17), P(18), L(18)]	A→C→H
12	R	13	2	15	[O(15), N(17), Q(17), M(17), T(17), P(18), L(18)]	A→B→F→K→R

13	O	14	1	15	[N(17), Q(17), M(17), T(17), P(18), L(18)]	$A \rightarrow C \rightarrow H \rightarrow O$
14	Q	13	4	17	[N(17), M(17), T(17), P(18), L(18)]	$A \rightarrow B \rightarrow E \rightarrow J \rightarrow Q$
15	N	13	4	17	[M(17), T(17), P(18), L(18)]	$A \rightarrow C \rightarrow H \rightarrow N$
16	M	14	3	17	[S(17), T(17), P(18), L(18)]	$A \rightarrow C \rightarrow G \rightarrow M$
17	S	16	1	17	[T(17), P(18), L(18)]	$A \rightarrow C \rightarrow G \rightarrow M \rightarrow S$
18	T	17	0	17	[P(18), L(18)]	$A \rightarrow B \rightarrow F \rightarrow K \rightarrow R \rightarrow T$

Resultado A*:

- Ordem de expansao: $[A, B, C, E, D, I, F, K, G, J, H, R, O, Q, N, M, S, T]$.
- Caminho solucao: $A \rightarrow B \rightarrow F \rightarrow K \rightarrow R \rightarrow T$.
- Profundidade: 5.
- Custo final: 17.
- Nos gerados: 20.
- Nos expandidos: 18.

2.5 Experimento: alterar heuristica de exatamente 3 nos

Alteracoes aplicadas:

No	h original	h novo
B	8	2
H	4	8
G	6	10

Table 10: Heurísticas alteradas no experimento

Reexecucao com heuristica modificada:

Algoritmo	Caminho	Custo	Gerados	Expandidos	Observacao
Greedy (original)	$A \rightarrow C \rightarrow H \rightarrow O \rightarrow T$	19	9	5	nao otimo
Greedy (modificado)	$A \rightarrow B \rightarrow F \rightarrow K \rightarrow R \rightarrow T$	17	10	6	passou a otimo
A* (original)	$A \rightarrow B \rightarrow F \rightarrow K \rightarrow R \rightarrow T$	17	20	18	otimo
A* (modificado)	$A \rightarrow B \rightarrow F \rightarrow K \rightarrow R \rightarrow T$	17	16	12	mesmo otimo, menos expansoes

Table 11: Impacto da alteracao de heurísticas

Arvores do experimento (heuristica modificada):

Greedy modificado:

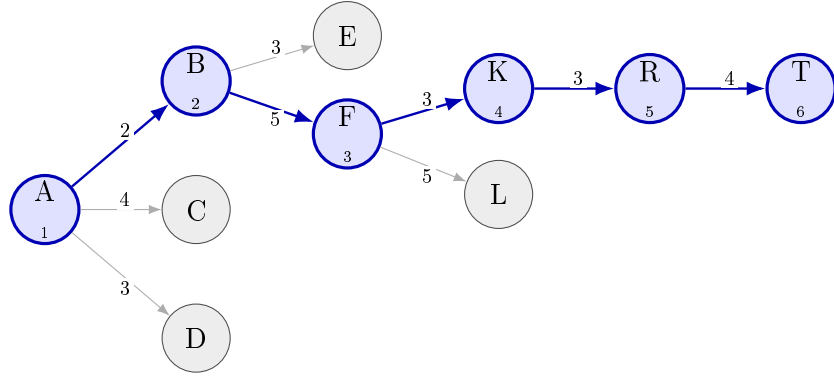


Figure 12: Arvore do Greedy com heuristica modificada (azul = caminho solucao)

A modificado:*

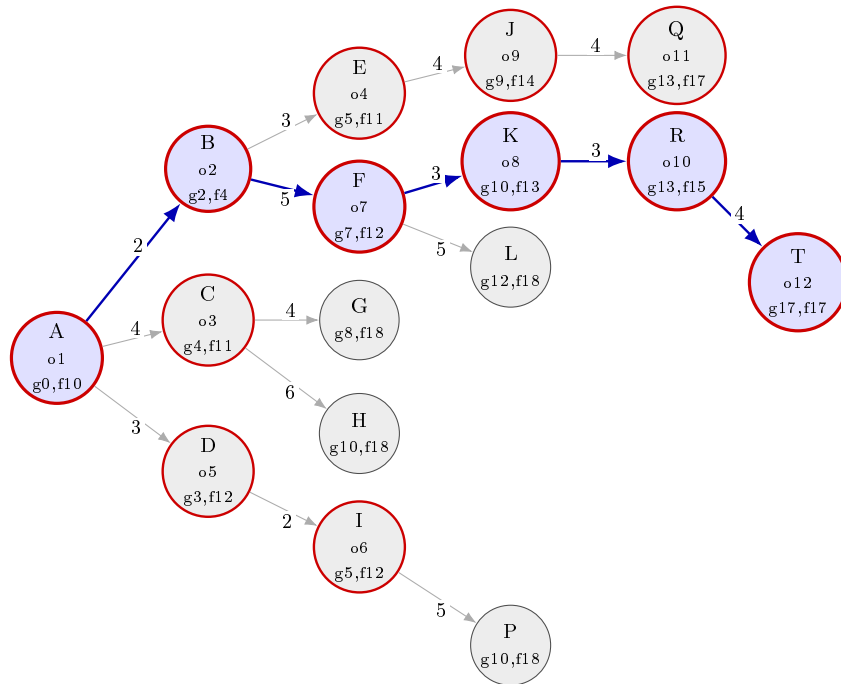


Figure 13: Arvore do A* com heuristica modificada (azul = caminho solucao; borda vermelha = no visitado)

2.6 Respostas objetivas do experimento

- a) A solucao encontrada mudou? **Sim no Greedy** (mudou para o caminho otimo). **No A* nao.**
- b) O custo da solucao mudou? **Sim no Greedy** ($19 \rightarrow 17$). **No A* nao** (17 em ambos).
- c) A busca gulosa encontrou solucao otima? **Antes: nao. Depois: sim** (neste experimento).
- d) O A* encontrou solucao otima? **Sim**, antes e depois.

- e) Qual algoritmo expandiu menos nós? **Greedy** (5 no original; 6 no modificado), contra 18 e 12 do A*.
- f) Qual foi mais sensível a heurística? **Greedy**, pois alterou caminho e custo; no A* mudou principalmente eficiência.

2.7 Comparação final: Greedy vs A*

Notação:

b = fator de ramificação, d = profundidade da solução, m = profundidade máxima.

- Completeness:** Greedy não é completo em geral; A* é completo em condições usuais (custos positivos e espaço finito/adequado).
- Optimality:** Greedy não garante ótimo; A* garante ótimo com heurística admissível/consistente.
- Consumo de memória:** ambos guardam fronteira ordenada; A* tende a manter mais estados por considerar custo acumulado.
- Dependência da heurística:** Greedy é altamente dependente; A* também depende, mas de forma mais robusta.
- Custo computacional:** Greedy costuma expandir menos quando a heurística aponta bem; A* expande mais, mas com melhor garantia.
- Qualidade da solução:** A* superior em garantia de qualidade; Greedy pode ser bom, mas pode errar caminho.
- Comportamento em grafos grandes:** Greedy pode ser rápido, porém arriscado; A* é mais confiável, mas pode consumir muita memória.

3 Questão 3 - Busca Local (8-Rainhas)

3.1 Formulação em espaço de estados

$$P = \langle Z, z_0, \text{Goal}, \text{Succ}, c \rangle$$

- a) Estado formal: $z = (s_1, \dots, s_8)$, com $s_i \in \{1, \dots, 8\}$ indicando a linha da rainha na coluna i . Assim, $Z = \{1, \dots, 8\}^8$.
- b) Estado inicial: $z_0 = (1, 1, 1, 1, 1, 1, 1, 1)$.
- c) Teste de objetivo: $\text{Goal}(z)$ verdadeiro sse $h(z) = 0$, onde $h(z)$ é o número de pares de rainhas em conflito (mesma linha ou diagonal).
- d) Função sucessora: $\text{Succ}(z)$ gera todos os estados obtidos movendo exatamente uma rainha para outra linha na mesma coluna.
- e) Função de custo: $c(z, z') = 1$ por movimento local (vizinho); em busca local, minimiza-se principalmente $h(z)$.

- f) Representacao em Python: tupla de 8 inteiros (ex.: (1,1,1,1,1,1,1,1)).

Interpretacao da superficie de busca (local):

- A paisagem de $h(s)$ e discreta e multimodal, com muitos planaltos e minimos locais.
- Movimentos de uma unica rainha por coluna podem reduzir conflitos localmente, mas tambem podem levar a regioes sem melhoria estrita.

3.2 Hill-Climbing (execucao manual)

Iteracao	Estado atual	$h(s)$	Estado escolhido e justificativa
0	[1,1,1,1,1,1,1,1]	28	[1,1,1,1,1,1,8,1] (h=21): melhor vizinho
1	[1,1,1,1,1,8,1,1]	21	[1,1,1,1,1,1,8,2] (h=15): melhor vizinho
2	[1,1,1,1,1,8,2,1]	15	[1,1,1,1,7,1,8,2] (h=10): melhor vizinho
3	[1,1,1,1,7,1,8,2]	10	[1,1,6,1,7,1,8,2] (h=6): melhor vizinho
4	[1,1,6,1,7,1,8,2]	6	[5,1,6,1,7,1,8,2] (h=3): melhor vizinho
5	[5,1,6,1,7,1,8,2]	3	[5,1,6,4,7,1,8,2] (h=1): melhor vizinho
6	[5,1,6,4,7,1,8,2]	1	parada: nao existe vizinho com h menor

Resultado do HC manual:

- Estado final: [5,1,6,4,7,1,8,2].
- h final: 1.
- Nao encontrou solucao valida ($h \neq 0$).
- Numero total de iteracoes executadas: 7 (iteracoes 0 a 6), com 6 melhorias e 1 parada.
- Tipo de parada: **plato** (existem vizinhos com mesmo valor de h , mas nenhum melhor).

3.3 Top-5 vizinhos por iteracao (HC)

Iteracao	Vizinho	$h(s)$
0	[1,1,1,1,1,1,8,1]	21
0	[1,1,1,1,1,7,1,1]	21
0	[1,1,1,1,1,8,1,1]	21
0	[1,1,1,1,6,1,1,1]	21
0	[1,1,1,1,7,1,1,1]	21
1	[1,1,1,1,1,1,8,2]	15
1	[1,1,1,1,7,1,8,1]	15
1	[1,1,1,6,1,1,8,1]	15
1	[1,1,1,7,1,1,8,1]	15
1	[1,1,5,1,1,1,8,1]	15
2	[1,1,1,1,7,1,8,2]	10
2	[1,1,1,7,1,1,8,2]	10
2	[1,1,5,1,1,1,8,2]	10
2	[1,1,6,1,1,1,8,2]	10
2	[1,6,1,1,1,1,8,2]	10
3	[1,1,6,1,7,1,8,2]	6
3	[1,6,1,1,7,1,8,2]	6
3	[5,1,1,1,7,1,8,2]	6
3	[1,1,1,1,7,2,8,2]	7
3	[1,1,1,1,7,3,8,2]	7
4	[5,1,6,1,7,1,8,2]	3

4	[1,1,6,1,7,2,8,2]	4
4	[1,1,6,1,7,4,8,2]	4
4	[1,1,6,1,7,5,8,2]	4
4	[1,1,6,2,7,1,8,2]	4
5	[5,1,6,4,7,1,8,2]	1
5	[5,1,6,1,7,2,8,2]	2
5	[5,1,6,1,7,4,8,2]	2
5	[5,2,6,1,7,1,8,2]	2
5	[5,1,6,1,3,1,8,2]	3
6	[5,1,6,4,7,3,8,2]	1
6	[5,3,6,4,7,1,8,2]	1
6	[3,1,6,4,7,1,8,2]	2
6	[5,1,6,4,3,1,8,2]	2
6	[5,1,6,4,7,1,3,2]	2

3.4 Todos os vizinhos avaliados por iteracao (HC)

Iteracao	Estado atual	Vizinho avaliado	$h(s)$
0	[1,1,1,1,1,1,1]	[1,1,1,1,1,1,8,1]	21
		[1,1,1,1,1,7,1,1]	21
		[1,1,1,1,1,8,1,1]	21
		[1,1,1,1,6,1,1,1]	21
		[1,1,1,1,7,1,1,1]	21
		[1,1,1,1,8,1,1,1]	21
		[1,1,1,6,1,1,1,1]	21
		[1,1,1,7,1,1,1,1]	21
		[1,1,1,8,1,1,1,1]	21
		[1,1,7,1,1,1,1,1]	21
		[1,1,8,1,1,1,1,1]	21
		[1,8,1,1,1,1,1,1]	21
		[1,1,1,1,1,1,1,2]	22
		[1,1,1,1,1,1,1,3]	22
		[1,1,1,1,1,1,1,4]	22
		[1,1,1,1,1,1,1,5]	22
		[1,1,1,1,1,1,1,6]	22
		[1,1,1,1,1,1,1,7]	22
		[1,1,1,1,1,1,1,8]	22
		[1,1,1,1,1,1,3,1]	22
		[1,1,1,1,1,1,4,1]	22
		[1,1,1,1,1,1,5,1]	22
		[1,1,1,1,1,1,6,1]	22
		[1,1,1,1,1,1,7,1]	22
		[1,1,1,1,1,1,4,1,1]	22
		[1,1,1,1,1,1,5,1,1]	22
		[1,1,1,1,1,1,6,1,1]	22
		[1,1,1,1,1,5,1,1,1]	22
		[1,1,1,5,1,1,1,1,1]	22
		[1,1,4,1,1,1,1,1,1]	22
		[1,1,5,1,1,1,1,1,1]	22
		[1,1,6,1,1,1,1,1,1]	22
		[1,3,1,1,1,1,1,1,1]	22
		[1,4,1,1,1,1,1,1,1]	22
		[1,5,1,1,1,1,1,1,1]	22
		[1,6,1,1,1,1,1,1,1]	22
		[1,7,1,1,1,1,1,1,1]	22
		[2,1,1,1,1,1,1,1,1]	22
		[3,1,1,1,1,1,1,1,1]	22
		[4,1,1,1,1,1,1,1,1]	22
		[5,1,1,1,1,1,1,1,1]	22
		[6,1,1,1,1,1,1,1,1]	22
		[7,1,1,1,1,1,1,1,1]	22
		[8,1,1,1,1,1,1,1,1]	22
		[1,1,1,1,1,1,1,2,1]	23
		[1,1,1,1,1,1,2,1,1]	23

		[1,1,1,1,1,3,1,1]	23
		[1,1,1,1,2,1,1,1]	23
		[1,1,1,1,3,1,1,1]	23
		[1,1,1,1,4,1,1,1]	23
		[1,1,1,2,1,1,1,1]	23
		[1,1,1,3,1,1,1,1]	23
		[1,1,1,4,1,1,1,1]	23
		[1,1,2,1,1,1,1,1]	23
		[1,1,3,1,1,1,1,1]	23
		[1,2,1,1,1,1,1,1]	23
1	[1,1,1,1,1,1,8,1]	[1,1,1,1,1,1,8,2]	15
		[1,1,1,1,7,1,8,1]	15
		[1,1,1,6,1,1,8,1]	15
		[1,1,1,7,1,1,8,1]	15
		[1,1,5,1,1,1,8,1]	15
		[1,1,7,1,1,1,8,1]	15
		[1,6,1,1,1,1,8,1]	15
		[7,1,1,1,1,1,8,1]	15
		[1,1,1,1,1,1,8,3]	16
		[1,1,1,1,1,1,8,4]	16
		[1,1,1,1,1,1,8,5]	16
		[1,1,1,1,1,1,8,6]	16
		[1,1,1,1,1,2,8,1]	16
		[1,1,1,1,1,4,8,1]	16
		[1,1,1,1,1,5,8,1]	16
		[1,1,1,1,1,6,8,1]	16
		[1,1,1,1,1,7,8,1]	16
		[1,1,1,1,1,8,8,1]	16
		[1,1,1,1,3,1,8,1]	16
		[1,1,1,1,5,1,8,1]	16
		[1,1,1,1,6,1,8,1]	16
		[1,1,1,1,8,1,8,1]	16
		[1,1,1,4,1,1,8,1]	16
		[1,1,1,8,1,1,8,1]	16
		[1,1,6,1,1,1,8,1]	16
		[1,1,8,1,1,1,8,1]	16
		[1,4,1,1,1,1,8,1]	16
		[1,5,1,1,1,1,8,1]	16
		[1,7,1,1,1,1,8,1]	16
		[1,8,1,1,1,1,8,1]	16
		[3,1,1,1,1,1,8,1]	16
		[4,1,1,1,1,1,8,1]	16
		[5,1,1,1,1,1,8,1]	16
		[6,1,1,1,1,1,8,1]	16
		[1,1,1,1,1,1,8,7]	17
		[1,1,1,1,1,1,8,8]	17
		[1,1,1,1,1,3,8,1]	17
		[1,1,1,1,2,1,8,1]	17
		[1,1,1,1,4,1,8,1]	17
		[1,1,1,2,1,1,8,1]	17
		[1,1,1,3,1,1,8,1]	17
		[1,1,1,5,1,1,8,1]	17
		[1,1,2,1,1,1,8,1]	17
		[1,1,3,1,1,1,8,1]	17
		[1,1,4,1,1,1,8,1]	17
		[1,2,1,1,1,1,8,1]	17
		[1,3,1,1,1,1,8,1]	17
		[2,1,1,1,1,1,8,1]	17
		[8,1,1,1,1,1,8,1]	17
		[1,1,1,1,1,1,3,1]	22
		[1,1,1,1,1,1,4,1]	22
		[1,1,1,1,1,1,5,1]	22
		[1,1,1,1,1,1,6,1]	22
		[1,1,1,1,1,1,7,1]	22
		[1,1,1,1,1,1,2,1]	23
		[1,1,1,1,1,1,1,1]	28
2	[1,1,1,1,1,1,8,2]	[1,1,1,1,7,1,8,2]	10

	[1,1,1,7,1,1,8,2]	10
	[1,1,5,1,1,1,8,2]	10
	[1,1,6,1,1,1,8,2]	10
	[1,6,1,1,1,1,8,2]	10
	[1,7,1,1,1,1,8,2]	10
	[7,1,1,1,1,1,8,2]	10
	[1,1,1,1,1,3,8,2]	11
	[1,1,1,1,1,5,8,2]	11
	[1,1,1,1,1,6,8,2]	11
	[1,1,1,1,1,7,8,2]	11
	[1,1,1,1,1,8,8,2]	11
	[1,1,1,1,3,1,8,2]	11
	[1,1,1,1,4,1,8,2]	11
	[1,1,1,1,6,1,8,2]	11
	[1,1,1,1,8,1,8,2]	11
	[1,1,1,4,1,1,8,2]	11
	[1,1,1,5,1,1,8,2]	11
	[1,1,1,6,1,1,8,2]	11
	[1,1,1,8,1,1,8,2]	11
	[1,1,7,1,1,1,8,2]	11
	[1,1,8,1,1,1,8,2]	11
	[1,4,1,1,1,1,8,2]	11
	[1,5,1,1,1,1,8,2]	11
	[3,1,1,1,1,1,8,2]	11
	[4,1,1,1,1,1,8,2]	11
	[5,1,1,1,1,1,8,2]	11
	[6,1,1,1,1,1,8,2]	11
	[8,1,1,1,1,1,8,2]	11
	[1,1,1,1,1,2,8,2]	12
	[1,1,1,1,1,4,8,2]	12
	[1,1,1,1,5,1,8,2]	12
	[1,1,1,3,1,1,8,2]	12
	[1,1,3,1,1,1,8,2]	12
	[1,1,4,1,1,1,8,2]	12
	[1,3,1,1,1,1,8,2]	12
	[1,8,1,1,1,1,8,2]	12
	[1,1,1,1,2,1,8,2]	13
	[1,1,1,2,1,1,8,2]	13
	[1,1,2,1,1,1,8,2]	13
	[1,2,1,1,1,1,8,2]	13
	[2,1,1,1,1,1,8,2]	13
	[1,1,1,1,1,1,4,2]	16
	[1,1,1,1,1,1,5,2]	16
	[1,1,1,1,1,1,6,2]	16
	[1,1,1,1,1,1,7,2]	16
	[1,1,1,1,1,1,8,3]	16
	[1,1,1,1,1,1,8,4]	16
	[1,1,1,1,1,1,8,5]	16
	[1,1,1,1,1,1,8,6]	16
	[1,1,1,1,1,1,2,2]	17
	[1,1,1,1,1,1,3,2]	17
	[1,1,1,1,1,1,8,7]	17
	[1,1,1,1,1,1,8,8]	17
	[1,1,1,1,1,1,8,1]	21
	[1,1,1,1,1,1,1,2]	22
3	[1,1,1,1,7,1,8,2]	6
	[1,6,1,1,7,1,8,2]	6
	[5,1,1,1,7,1,8,2]	6
	[1,1,1,1,7,2,8,2]	7
	[1,1,1,1,7,3,8,2]	7
	[1,1,1,1,7,5,8,2]	7
	[1,1,1,4,7,1,8,2]	7
	[1,1,1,5,7,1,8,2]	7
	[1,1,1,7,7,1,8,2]	7
	[1,1,3,1,7,1,8,2]	7
	[1,1,5,1,7,1,8,2]	7
	[1,1,8,1,7,1,8,2]	7

	[1,4,1,1,7,1,8,2]	7
	[1,5,1,1,7,1,8,2]	7
	[1,7,1,1,7,1,8,2]	7
	[4,1,1,1,7,1,8,2]	7
	[6,1,1,1,7,1,8,2]	7
	[7,1,1,1,7,1,8,2]	7
	[8,1,1,1,7,1,8,2]	7
	[1,1,1,1,7,4,8,2]	8
	[1,1,1,1,7,6,8,2]	8
	[1,1,1,1,7,7,8,2]	8
	[1,1,1,1,7,8,8,2]	8
	[1,1,1,2,7,1,8,2]	8
	[1,1,1,3,7,1,8,2]	8
	[1,1,1,6,7,1,8,2]	8
	[1,1,1,8,7,1,8,2]	8
	[1,1,4,1,7,1,8,2]	8
	[1,1,7,1,7,1,8,2]	8
	[1,3,1,1,7,1,8,2]	8
	[1,8,1,1,7,1,8,2]	8
	[3,1,1,1,7,1,8,2]	8
	[1,1,2,1,7,1,8,2]	9
	[1,2,1,1,7,1,8,2]	9
	[2,1,1,1,7,1,8,2]	9
	[1,1,1,1,3,1,8,2]	11
	[1,1,1,1,4,1,8,2]	11
	[1,1,1,1,6,1,8,2]	11
	[1,1,1,1,7,1,3,2]	11
	[1,1,1,1,7,1,4,2]	11
	[1,1,1,1,7,1,6,2]	11
	[1,1,1,1,7,1,8,3]	11
	[1,1,1,1,7,1,8,4]	11
	[1,1,1,1,7,1,8,5]	11
	[1,1,1,1,7,1,8,6]	11
	[1,1,1,1,8,1,8,2]	11
	[1,1,1,1,5,1,8,2]	12
	[1,1,1,1,7,1,2,2]	12
	[1,1,1,1,7,1,5,2]	12
	[1,1,1,1,7,1,7,2]	12
	[1,1,1,1,7,1,8,8]	12
	[1,1,1,1,2,1,8,2]	13
	[1,1,1,1,7,1,8,7]	13
	[1,1,1,1,1,1,8,2]	15
	[1,1,1,1,7,1,8,1]	15
	[1,1,1,1,7,1,1,2]	16
4	[1,1,6,1,7,1,8,2]	3
	[1,1,6,1,7,2,8,2]	4
	[1,1,6,1,7,4,8,2]	4
	[1,1,6,1,7,5,8,2]	4
	[1,1,6,2,7,1,8,2]	4
	[1,1,6,4,7,1,8,2]	4
	[1,4,6,1,7,1,8,2]	4
	[1,6,6,1,7,1,8,2]	4
	[3,1,6,1,7,1,8,2]	4
	[7,1,6,1,7,1,8,2]	4
	[1,1,6,1,7,3,8,2]	5
	[1,1,6,1,7,7,8,2]	5
	[1,1,6,1,7,8,8,2]	5
	[1,1,6,3,7,1,8,2]	5
	[1,1,6,5,7,1,8,2]	5
	[1,1,6,7,7,1,8,2]	5
	[1,1,6,8,7,1,8,2]	5
	[1,2,6,1,7,1,8,2]	5
	[1,3,6,1,7,1,8,2]	5
	[1,5,6,1,7,1,8,2]	5
	[1,7,6,1,7,1,8,2]	5
	[1,8,6,1,7,1,8,2]	5
	[4,1,6,1,7,1,8,2]	5

	[6,1,6,1,7,1,8,2]	5
	[8,1,6,1,7,1,8,2]	5
	[1,1,6,1,3,1,8,2]	6
	[1,1,6,1,7,6,8,2]	6
	[1,1,6,6,7,1,8,2]	6
	[2,1,6,1,7,1,8,2]	6
	[1,1,3,1,7,1,8,2]	7
	[1,1,5,1,7,1,8,2]	7
	[1,1,6,1,7,1,3,2]	7
	[1,1,6,1,7,1,4,2]	7
	[1,1,6,1,7,1,5,2]	7
	[1,1,6,1,7,1,8,3]	7
	[1,1,6,1,7,1,8,4]	7
	[1,1,6,1,7,1,8,5]	7
	[1,1,6,1,7,1,8,6]	7
	[1,1,8,1,7,1,8,2]	7
	[1,1,4,1,7,1,8,2]	8
	[1,1,6,1,4,1,8,2]	8
	[1,1,6,1,5,1,8,2]	8
	[1,1,6,1,6,1,8,2]	8
	[1,1,6,1,7,1,6,2]	8
	[1,1,6,1,7,1,7,2]	8
	[1,1,6,1,7,1,8,8]	8
	[1,1,6,1,8,1,8,2]	8
	[1,1,7,1,7,1,8,2]	8
	[1,1,2,1,7,1,8,2]	9
	[1,1,6,1,2,1,8,2]	9
	[1,1,6,1,7,1,2,2]	9
	[1,1,6,1,7,1,8,7]	9
	[1,1,1,1,7,1,8,2]	10
	[1,1,6,1,1,1,8,2]	10
	[1,1,6,1,7,1,1,2]	11
	[1,1,6,1,7,1,8,1]	11
5	[5,1,6,1,7,1,8,2]	1
	[5,1,6,4,7,1,8,2]	1
	[5,1,6,1,7,2,8,2]	2
	[5,1,6,1,7,4,8,2]	2
	[5,2,6,1,7,1,8,2]	2
	[5,1,6,1,3,1,8,2]	3
	[5,1,6,1,7,3,8,2]	3
	[5,1,6,1,7,5,8,2]	3
	[5,1,6,1,7,6,8,2]	3
	[5,1,6,1,7,7,8,2]	3
	[5,1,6,1,7,8,8,2]	3
	[5,1,6,2,7,1,8,2]	3
	[5,1,6,3,7,1,8,2]	3
	[5,1,6,7,7,1,8,2]	3
	[5,3,6,1,7,1,8,2]	3
	[5,4,6,1,7,1,8,2]	3
	[5,6,6,1,7,1,8,2]	3
	[5,7,6,1,7,1,8,2]	3
	[5,8,6,1,7,1,8,2]	3
	[3,1,6,1,7,1,8,2]	4
	[5,1,3,1,7,1,8,2]	4
	[5,1,6,1,7,1,3,2]	4
	[5,1,6,1,7,1,4,2]	4
	[5,1,6,1,7,1,7,2]	4
	[5,1,6,1,7,1,8,3]	4
	[5,1,6,1,7,1,8,4]	4
	[5,1,6,1,7,1,8,6]	4
	[5,1,6,1,7,1,8,8]	4
	[5,1,6,5,7,1,8,2]	4
	[5,1,6,6,7,1,8,2]	4
	[5,1,6,8,7,1,8,2]	4
	[5,1,8,1,7,1,8,2]	4
	[5,5,6,1,7,1,8,2]	4
	[7,1,6,1,7,1,8,2]	4
	[4,1,6,1,7,1,8,2]	5

	[5,1,4,1,7,1,8,2]	5
	[5,1,5,1,7,1,8,2]	5
	[5,1,6,1,4,1,8,2]	5
	[5,1,6,1,5,1,8,2]	5
	[5,1,6,1,6,1,8,2]	5
	[5,1,6,1,7,1,5,2]	5
	[5,1,6,1,7,1,6,2]	5
	[5,1,6,1,7,1,8,5]	5
	[5,1,6,1,8,1,8,2]	5
	[6,1,6,1,7,1,8,2]	5
	[8,1,6,1,7,1,8,2]	5
	[1,1,6,1,7,1,8,2]	6
	[2,1,6,1,7,1,8,2]	6
	[5,1,1,1,7,1,8,2]	6
	[5,1,2,1,7,1,8,2]	6
	[5,1,6,1,2,1,8,2]	6
	[5,1,6,1,7,1,2,2]	6
	[5,1,6,1,7,1,8,7]	6
	[5,1,7,1,7,1,8,2]	6
	[5,1,6,1,1,1,8,2]	7
	[5,1,6,1,7,1,1,2]	7
	[5,1,6,1,7,1,8,1]	7
6	[5,1,6,4,7,1,8,2]	1
	[5,3,6,4,7,1,8,2]	1
	[3,1,6,4,7,1,8,2]	2
	[5,1,6,4,3,1,8,2]	2
	[5,1,6,4,7,1,3,2]	2
	[5,1,6,4,7,1,4,2]	2
	[5,1,6,4,7,1,8,3]	2
	[5,1,6,4,7,1,8,5]	2
	[5,1,6,4,7,1,8,6]	2
	[5,1,6,4,7,2,8,2]	2
	[5,1,6,4,7,4,8,2]	2
	[5,1,6,4,7,5,8,2]	2
	[5,1,6,4,7,7,8,2]	2
	[5,1,6,4,7,8,8,2]	2
	[5,1,8,4,7,1,8,2]	2
	[5,2,6,4,7,1,8,2]	2
	[5,7,6,4,7,1,8,2]	2
	[5,8,6,4,7,1,8,2]	2
	[4,1,6,4,7,1,8,2]	3
	[5,1,1,4,7,1,8,2]	3
	[5,1,2,4,7,1,8,2]	3
	[5,1,3,4,7,1,8,2]	3
	[5,1,6,1,7,1,8,2]	3
	[5,1,6,2,7,1,8,2]	3
	[5,1,6,3,7,1,8,2]	3
	[5,1,6,4,2,1,8,2]	3
	[5,1,6,4,6,1,8,2]	3
	[5,1,6,4,7,1,5,2]	3
	[5,1,6,4,7,1,6,2]	3
	[5,1,6,4,7,1,7,2]	3
	[5,1,6,4,7,1,8,4]	3
	[5,1,6,4,7,1,8,8]	3
	[5,1,6,4,7,6,8,2]	3
	[5,1,6,4,8,1,8,2]	3
	[5,1,6,7,7,1,8,2]	3
	[5,4,6,4,7,1,8,2]	3
	[5,5,6,4,7,1,8,2]	3
	[5,6,6,4,7,1,8,2]	3
	[6,1,6,4,7,1,8,2]	3
	[7,1,6,4,7,1,8,2]	3
	[8,1,6,4,7,1,8,2]	3
	[1,1,6,4,7,1,8,2]	4
	[2,1,6,4,7,1,8,2]	4
	[5,1,4,4,7,1,8,2]	4
	[5,1,5,4,7,1,8,2]	4

[5,1,6,4,1,1,8,2]	4
[5,1,6,4,4,1,8,2]	4
[5,1,6,4,5,1,8,2]	4
[5,1,6,4,7,1,2,2]	4
[5,1,6,4,7,1,8,1]	4
[5,1,6,4,7,1,8,7]	4
[5,1,6,5,7,1,8,2]	4
[5,1,6,6,7,1,8,2]	4
[5,1,6,8,7,1,8,2]	4
[5,1,7,4,7,1,8,2]	4
[5,1,6,4,7,1,1,2]	5

3.5 Analise de armadilha de busca

- Tipo observado: **plato** (na iteracao final, melhor vizinho possui o mesmo valor de h).
- Por que ocorreu: a vizinhanca de um movimento por coluna cria regioes planas no espaco de busca.
- Impacto: o HC puro para sem atingir $h = 0$, mesmo estando proximo da solucao.
- Como evitar: random restart, movimentos laterais limitados ou simulated annealing.

3.6 Random Restart Hill-Climbing (20 execucoes)

Execucao	Estado inicial	Passos	$h(s)$ final	Solucao?
1	[2,6,2,4,7,8,8,4]	4	1	nao
2	[1,2,2,5,2,8,1,8]	3	1	nao
3	[6,4,7,5,6,6,7,2]	4	1	nao
4	[6,2,5,5,8,3,5,1]	1	3	nao
5	[6,6,8,7,2,7,8,2]	5	0	sim
6	[7,8,7,5,7,8,1,4]	3	1	nao
7	[3,1,2,7,8,3,8,4]	3	0	sim
8	[2,8,8,5,7,5,8,7]	4	1	nao
9	[7,4,5,5,1,1,3,8]	2	1	nao
10	[8,6,4,5,8,8,8,7]	3	1	nao
11	[1,5,6,8,2,8,5,8]	3	1	nao
12	[3,7,8,2,5,8,6,2]	2	2	nao
13	[4,7,3,8,1,2,1,8]	2	1	nao
14	[2,4,4,2,1,5,6,3]	4	0	sim
15	[1,6,8,3,4,8,3,5]	1	1	nao
16	[2,7,3,3,5,7,7,3]	4	2	nao
17	[4,7,1,2,7,2,1,6]	3	1	nao
18	[6,1,1,8,6,5,8,6]	5	0	sim
19	[2,8,8,3,3,4,7,6]	3	0	sim
20	[8,4,2,7,1,4,3,1]	3	1	nao

Resumo RR-HC:

- Taxa de sucesso: $5/20 = 25.0\%$.
- Media de h final: 0.95.
- Media de passos por execucao: 3.10.

3.7 Simulated Annealing

$$P(\text{aceitar pior}) = e^{-\Delta E/T}$$

- Temperatura inicial: $T_0 = 25.0$.
- Resfriamento: $T \leftarrow \alpha T$ com $\alpha = 0.97$.
- Parada: $T < 0.001$ ou limite de 3000 passos.
- Execucao detalhada: inicio [1,6,8,1,5,2,8,8], melhor estado [3,6,2,7,1,4,8,5], **best** $h = 0$ (sucesso), 311 passos.

Exemplos de movimentos piores aceitos (execucao detalhada):

Step	$h_{atual} \rightarrow h_{novo}$	Δ	T	$e^{-\Delta/T}$	sorteio
5	5 $\rightarrow$ 8	+3	21.4684	0.8696	0.3689
6	8 $\rightarrow$ 12	+4	20.8243	0.8252	0.4443
13	6 $\rightarrow$ 7	+1	16.8257	0.9423	0.8105
14	7 $\rightarrow$ 8	+1	16.3209	0.9406	0.4154
19	4 $\rightarrow$ 5	+1	14.0153	0.9311	0.6297
22	4 $\rightarrow$ 5	+1	12.7914	0.9248	0.3176
23	5 $\rightarrow$ 7	+2	12.4077	0.8511	0.1797
24	7 $\rightarrow$ 9	+2	12.0354	0.8469	0.3187

Lote SA (20 execucoes):

Execucao	Estado inicial	Passos	best h	Solucao?
1	[5,1,6,3,4,6,1,2]	333	1	nao
2	[7,8,4,5,7,1,7,4]	333	1	nao
3	[6,3,1,7,7,8,1,7]	291	0	sim
4	[2,1,3,8,7,5,7,2]	173	0	sim
5	[4,5,3,8,7,3,7,1]	333	1	nao
6	[8,8,7,2,2,6,6,1]	333	1	nao
7	[2,1,8,3,4,5,1,2]	333	1	nao
8	[1,2,5,2,5,5,6,2]	333	1	nao
9	[2,8,8,3,3,6,6,6]	319	0	sim
10	[1,3,2,7,1,7,5,6]	333	1	nao
11	[2,6,5,2,3,5,3,2]	333	1	nao
12	[6,4,8,1,5,7,6,2]	333	1	nao
13	[2,2,7,1,2,6,2,5]	333	1	nao
14	[4,5,2,7,4,7,6,3]	110	0	sim
15	[7,8,7,3,4,5,4,6]	333	1	nao
16	[4,1,8,8,4,7,2,8]	333	1	nao
17	[5,6,5,5,1,4,4,4]	333	1	nao
18	[8,7,2,6,4,3,4,6]	333	1	nao
19	[2,8,5,4,7,3,8,8]	333	1	nao
20	[2,2,6,7,3,1,8,6]	333	1	nao

Resumo SA:

- Taxa de sucesso: $4/20 = 20.0\%$.
- Media de best h : 0.80.
- Media de passos por execucao: 311.05.
- Comparacao direta com HC simples: o HC manual parou em $h = 1$, enquanto o SA alcançou $h = 0$ na execucao detalhada e encontrou 4 solucoes validas em 20 execucoes.

3.8 Comparacao final (Q3)

Criterio	Hill-Climbing	RR Hill-Climbing	Simulated Annealing
Qualidade da solucao	Falhou no caso manual ($h = 1$)	25% de sucesso (5/20)	20% de sucesso (4/20), com melhor best h medio (0.80)
Velocidade de convergencia	Muito rapido (6 melhorias)	Rapido por tentativa (3.10 passos em media)	Mais lento (311.05 passos em media)
Sensibilidade ao estado inicial	Alta	Media	Media/baixa
Escapa de maximos locais/platos	Nao	Parcialmente (por reinicio)	Sim (probabilistico)
Custo computacional	Baixo	Medio	Alto
Estabilidade dos resultados	Deterministico para um estado inicial fixo	Variavel entre execucoes	Variavel e sensivel aos parametros de temperatura

Table 17: Comparacao dos metodos de busca local para 8-rainhas

4 Questao 4 - CSP e Otimizacoes de Busca

4.1 Modelagem do CSP

$$P = \langle X, D, C \rangle$$

- Variaveis: $X = \{T_1, T_2, T_3, T_4, T_5, T_6\}$.
- Dominio: $D(T_i) = \{A, B, C, D\}$ para todo i .
- Restricoes unarias: $T_2 \neq A$ e $T_5 \neq A$.

- Restricoes binarias:

$$T_i \neq T_{i+1}, \quad i = 1, \dots, 5$$

e a restricao adicional $((T_2 = C) \wedge (T_5 = C))$ proibida.

- Restricoes globais:
 - B deve aparecer em pelo menos um entre T_3 ou T_4 ;
 - medico D pode aparecer no maximo 2 vezes na escala.

4.2 Grafo de restricoes

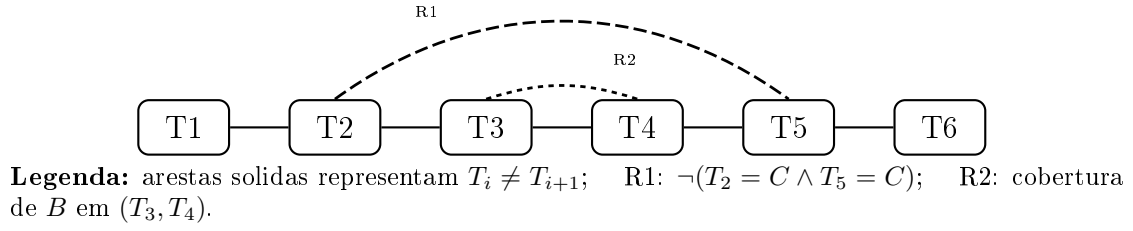


Figure 14: Grafo de restricoes usado na implementacao

4.3 Backtracking (manual + implementado)

Ordem fixa de atribuicao: $T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_4 \rightarrow T_5 \rightarrow T_6$.

Passo	Tentativa	Estado parcial	Resultado
1	$T_1 = A$	$\{T_1 = A\}$	segue
2	$T_2 = B$	$\{T_1 = A, T_2 = B\}$	segue
3	$T_3 = A$	$\{T_1 = A, T_2 = B, T_3 = A\}$	segue
4	$T_4 = A$	$\{T_1 = A, T_2 = B, T_3 = A, T_4 = A\}$	conflito
5	$T_4 = B$	$\{T_1 = A, T_2 = B, T_3 = A, T_4 = B\}$	segue
6	$T_5 = B$	$\{T_1 = A, T_2 = B, T_3 = A, T_4 = B, T_5 = B\}$	conflito
7	$T_5 = C$	$\{T_1 = A, T_2 = B, T_3 = A, T_4 = B, T_5 = C\}$	segue
8	$T_6 = A$	$\{T_1 = A, T_2 = B, T_3 = A, T_4 = B, T_5 = C, T_6 = A\}$	solucao

Estado final encontrado: (A, B, A, B, C, A) .

4.4 MRV + Degree

Com MRV (e desempate por Degree), a busca inicia por variaveis mais restritas (T_2, T_5). Nesta instancia, essa estrategia aumentou o numero de estados explorados em relacao ao BT de ordem fixa, mas ainda encontra a mesma solucao final.

4.5 Forward Checking

O Forward Checking elimina valores inviaveis de dominios assim que uma atribuicao e feita. Nos primeiros passos da execucao:

Passo	Atribuicao	Dominios removidos/reduzidos	Observacao
1	$T_2 = B$	remove B de T_1 e T_3	restricao de consecutividade
2	$T_5 = B$	remove B de T_4 e T_6	restricao de consecutividade

3	$T_3 = A$	cobertura de B forca $T_4 = \{B\}$, mas B ja removido de T_4	dominio vazio em T_4 (inconsistencia)
4	$T_3 = C$	mesma poda do passo 3	inconsistencia
5	$T_3 = D$	mesma poda do passo 3	inconsistencia
6	troca para $T_5 = C$	libera consistencia para T_3 e T_4	busca volta a crescer
7-10	$T_3 = A, T_4 = B, T_1 = A, T_6 = A$	apenas podas locais de consecutividade	solucao encontrada

Com isso, o FC detecta inconsistencias mais cedo e evita explorar subarvores inteiras que o BT simples ainda visitaria.

4.6 Backjumping

Nesta instancia, o Backjumping encontrou a solucao com o mesmo numero de estados do BT fixo e sem salto longo (saltos BJ = 0). Os conflitos identificados foram:

- $T_4 = A$ conflita com $T_3 = A$ (restricao de consecutividade);
- $T_5 = B$ conflita com $T_4 = B$ (restricao de consecutividade).

Como os conflitos foram sempre com a variavel pai imediata, nao houve oportunidade de backjump para ancestrais mais altos.

4.7 Comparacao quantitativa (Q4)

Todas as abordagens encontraram a mesma solucao: (A, B, A, B, C, A) .

Critério	BT	BT+MRV	BT+MRV+Degree	FC	BJ
Estados explorados	8	59	59	10	8
Backtracks	0	13	13	1	0
Profundidade maxima	6	6	6	6	6
Saltos BJ	0	0	0	0	0

Table 20: Resultados medidos pela implementacao autoral

4.8 Analise final pedida na questao

- Numero de estados explorados: melhor resultado em BT e BJ (8), seguido de FC (10).
- Numero de retrocessos: BT/BJ (0), FC (1), MRV e MRV+Degree (13).
- Velocidade de execucao: para esta instancia pequena, BT/BJ e FC foram mais rapidos.
- Consumo de memoria: todos baixos; FC usa mais controle de dominio, mas ainda leve neste tamanho.
- Facilidade de implementacao: $BT < MRV/Degree < FC < BJ$ (BJ e o mais trabalhoso).
- Eficiencia pratica: FC e BJ sao mais robustos para crescer a instancia, mesmo quando o ganho nesta instancia foi pequeno.

5 Questao 5 - Minimax e Poda Alpha-Beta

5.1 Conceitos pedidos no enunciado

- Objetivo do Minimax: escolher a acao que maximiza o resultado de MAX assumindo MIN otimo.
- Diferenca MAX/MIN: nos MAX escolhem maior valor; nos MIN escolhem menor valor.
- Utilidade: valor terminal (folha) que mede o resultado final do jogo.
- Propagacao: os valores sobem das folhas para a raiz alternando regras MAX/MIN.
- Adversario perfeito: MIN sempre joga da melhor forma para reduzir o ganho de MAX.
- Poda Alpha-Beta: elimina ramos que nao podem mais alterar a decisao final da raiz.

5.2 Dados da arvore

Folhas (esquerda para direita):

[3, 5, 6, 9, 1, 2, 0, -1, 7, 4, 5, 6]

Raiz A e do tipo MAX; filhos B, C, D sao MIN; cada um possui 4 folhas.

5.3 Minimax passo a passo

No	Tipo	Folhas/filhos avaliados	Valor
B	MIN	[3, 5, 6, 9]	3
C	MIN	[1, 2, 0, -1]	-1
D	MIN	[7, 4, 5, 6]	4
A	MAX	[$B = 3, C = -1, D = 4$]	4

Table 21: Propagacao de valores do Minimax

Decisao de MAX na raiz: escolher D (valor 4).

Ordem de expansao usada: esquerda $\rightarrow$ direita (B , depois C , depois D ; e folhas de cada no tambem da esquerda para direita).

Caminho escolhido: $A \rightarrow D$.

Nos visitados: 16 (4 internos + 12 folhas).

5.4 Arvore parcialmente preenchida (visao estrutural)

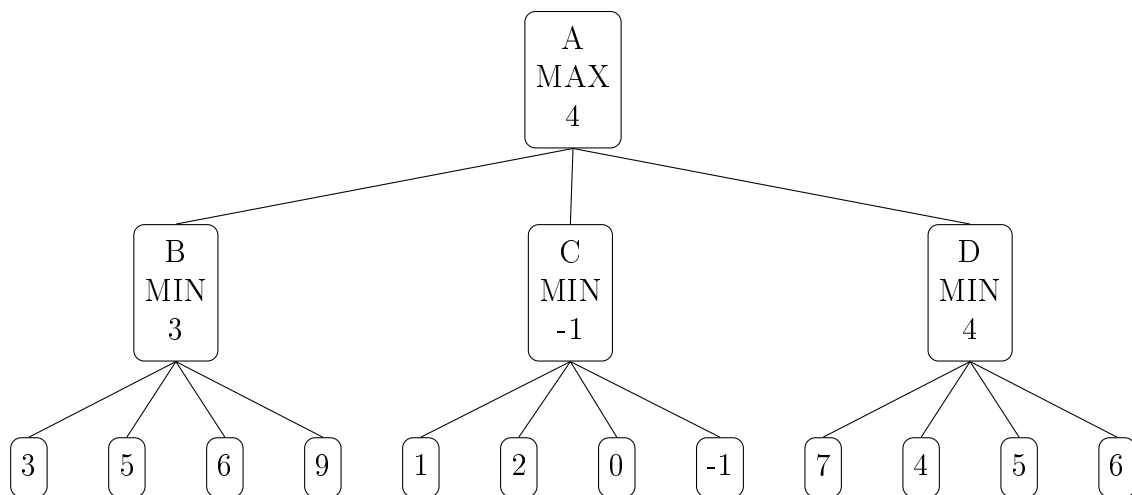


Figure 15: Arvore da questao com valores Minimax

5.5 Alpha-Beta (mesma ordem do enunciado)

Passo	No MIN	Folha	α	β	Poda
1	B	3	$-\infty$	3	nao
2	B	5	$-\infty$	3	nao
3	B	6	$-\infty$	3	nao
4	B	9	$-\infty$	3	nao
5	C	1	3	1	sim
6	D	7	3	7	nao
7	D	4	3	4	nao
8	D	5	3	4	nao
9	D	6	3	4	nao

Resultado:

- valor na raiz continua 4 (mesma decisao final do Minimax);
- folhas podadas: 3 (as 3 ultimas folhas de C);
- ramos podados: no subarvore de C , apos avaliar a folha 1;
- nos nao explorados: 3 folhas (2, 0 e -1);
- nos visitados: 13 (4 internos + 9 folhas).

Motivo matematico da poda: em um no MIN, quando $\beta \leq \alpha$, nenhum descendente restante pode melhorar a decisao de MAX na raiz.

Observacao importante: no registro do Alpha-Beta com poda, C aparece com valor parcial 1, porque o no foi cortado apos a primeira folha. Isso nao contradiz o Minimax completo ($C = -1$), apenas indica avaliacao interrompida pelo limite $\beta \leq \alpha$.

5.6 Representacao visual da poda Alpha-Beta

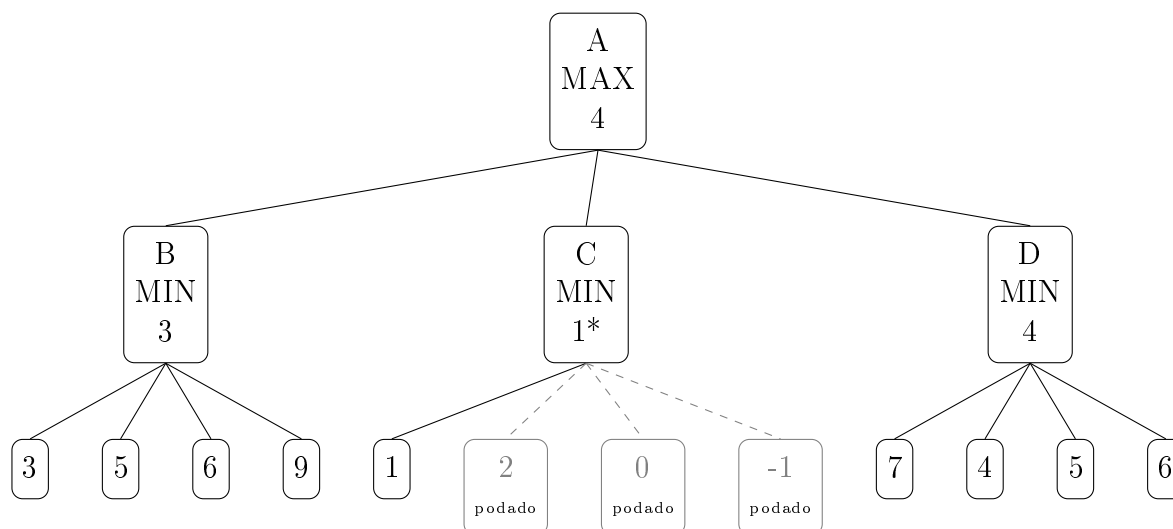


Figure 16: Poda Alpha-Beta na ordem original: em C , apenas a folha 1 é avaliada; as demais 3 folhas são podadas

5.7 Reordenacao para maximizar podas

Reordenacao usada na implementacao:

raiz: $[D, B, C]$

e, dentro de cada nó MIN, folhas em ordem crescente para provocar corte cedo.

- Podas antes da reordenacao: 3 folhas podadas.
- Podas depois da reordenacao: 6 folhas podadas.
- Valor final na raiz permanece 4 (decisao correta preservada).
- A nova ordem melhora eficiencia porque encontra limites fortes cedo, ativando cortes antecipados.
- Relacao ordenacao x desempenho: quanto melhor a ordem de exploracao (melhores nos primeiro), maior tende a ser o ganho da poda Alpha-Beta.

5.8 Minimax limitado (profundidade 2)

Heurísticas de corte fornecidas:

$[4, 7, 2, 5, 6, 1]$

Agrupando em 3 blocos MIN:

$$\min(4, 7) = 4, \quad \min(2, 5) = 2, \quad \min(6, 1) = 1$$

Logo, na raiz:

$$\max(4, 2, 1) = 4$$

Decisao do limitado coincide com o valor da raiz no Minimax completo (4). **Possiveis erros por heuristica:** se a estimativa local estiver enviesada, o Minimax limitado pode escolher um ramo diferente do Minimax completo, pois não enxerga efeitos táticos que só aparecem em profundidades maiores.

5.9 Experimento adicional: alterar 3 folhas

Alteracao aplicada:

$$[3, 5, 6] \rightarrow [8, 8, 8]$$

- Minimax modificado: raiz=8, $B = 8$, $C = -1$, $D = 4$.
- Alpha-Beta modificado: raiz=8, com 10 nos visitados e 6 folhas podadas.
- A decisao final muda: MAX passa de D para B .
- Sensibilidade as utilidades: pequena alteracao em folhas do ramo B alterou o valor minimax do ramo de 3 para 8, mudando totalmente a decisao da raiz.

5.10 Desenhos pos-experimento (Minimax e Alpha-Beta)

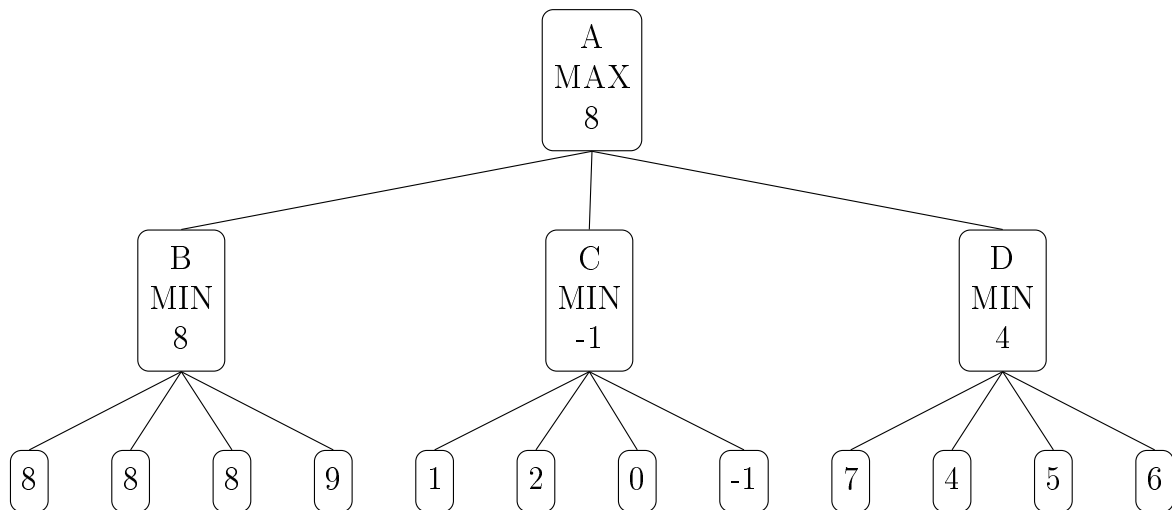


Figure 17: Arvore Minimax apos o experimento: MAX escolhe B (valor 8)

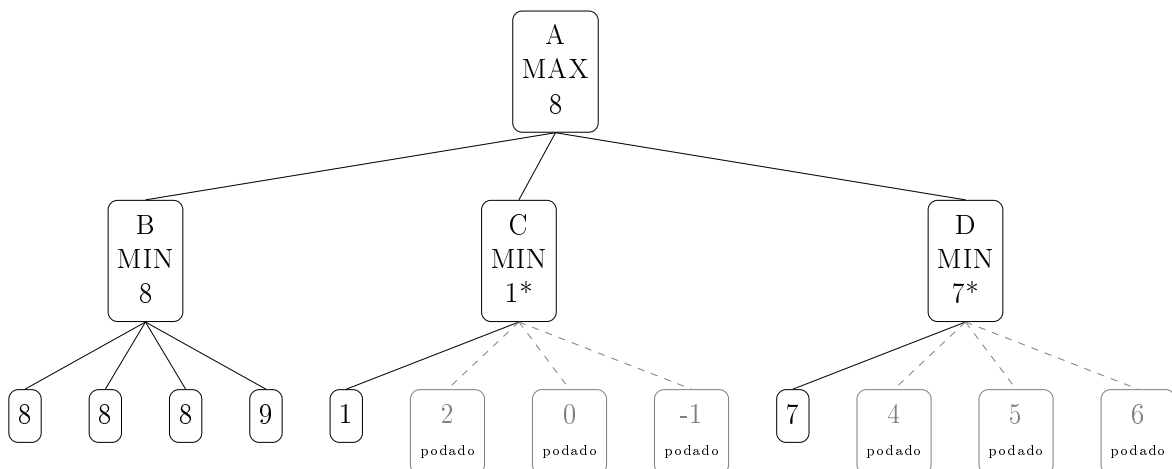


Figure 18: Alpha-Beta pos-experimento (ordem original): 6 podas (3 em C e 3 em D)

5.11 Comparacao final (Q5)

Cenario	Minimax (nos)	Alpha-Beta (nos / podas)
Original	16	13 / 3 folhas
Reordenado	-	10 / 6 folhas
Modificado (3 folhas)	16	10 / 6 folhas

Table 23: Comparacao quantitativa das execucoes

Criterio	Minimax	Alpha-Beta
Nos explorados (nesta prova)	16 nos no cenario original e 16 no modificado	13 nos no original; 10 com reordenacao e no modificado
Quantidade de podas	0 (nao realiza poda)	3 podas no original; 6 apos reordenacao
Custo computacional (teorico)	$O(b^m)$	pior caso $O(b^m)$; melhor caso com boa ordenacao $O(b^{m/2})$
Consumo de memoria (teorico)	$O(bm)$ (busca em profundidade)	$O(bm)$, mesma ordem assintotica
Impacto da ordenacao dos filhos	baixo (nao altera exploracao)	alto (ordem melhor aumenta as podas)
Qualidade da decisao	exata (adversario perfeito)	exata e igual ao Minimax quando a poda e correta

Table 24: Comparacao Minimax vs Alpha-Beta com criterios teoricos e evidencias da execucao

Sintese:

- custo computacional: Alpha-Beta $\leq$ Minimax (mesma decisao, menos nos);
- consumo de memoria: mesma ordem assintotica (DFS na arvore), diferindo pouco na pratica aqui;
- impacto da ordenacao: muito alto no Alpha-Beta (podas sobem de 3 para 6);
- qualidade da decisao: igual ao Minimax quando os cortes sao corretos.

6 Questao 6 - Monte Carlo Tree Search (MCTS)

6.1 Escopo e modelagem usada

Para a Q6, foi usada uma modelagem simplificada e autoral do estado de decisao na raiz do Connect-4:

- Acoes possiveis na raiz: $\{C1, C2, C3, C4\}$;
- cada acao gera um filho imediato na arvore de busca;

- cada simulacao retorna resultado binario para MAX (vermelho): vitoria = 1, derrota = 0.

Representacao em Python (`codes/q6_mcts.py`):

- no armazena `visits = N(s)` e `wins = W(s)`;
- estrutura de arvore por ponteiro `parent` + dicionario `children`;
- implementacao 100% autoral, sem bibliotecas externas de jogos/MCTS.

6.2 Explicacao do MCTS (itens teoricos do enunciado)

- **Selecao:** desce pela arvore escolhendo filho com maior UCT.
- **Expansao:** ao atingir no com filhos ainda nao criados, adiciona-se um novo filho.
- **Simulacao (rollout):** a partir do no expandido, simula-se uma partida ate termino (aqui, modelo simplificado de resultado).
- **Retropropagacao:** atualiza-se $N(s)$ e $W(s)$ do no atual ate a raiz.

Metricas pedidas:

- $N(s)$: numero de visitas do estado.
- $W(s)$: numero de vitorias acumuladas para MAX.
- Exploitation: $\frac{W(s)}{N(s)}$ (qualidade media observada).
- Exploration: termo que prioriza nos pouco visitados.

6.3 Formula UCT usada

$$UCT(s_i) = \frac{W(s_i)}{N(s_i)} + C \sqrt{\frac{\ln N(\text{pai})}{N(s_i)}}$$

com $C = 1.4$ no experimento principal.

6.4 Execucao detalhada (10 iteracoes)

Iter.	Caminho	No pandido/reusado	ex- Rollout	(N, W) no no escolhido
1	raiz $\rightarrow C1$	$C1$	vitoria	(1, 1)
2	raiz $\rightarrow C2$	$C2$	vitoria	(1, 1)
3	raiz $\rightarrow C3$	$C3$	vitoria	(1, 1)
4	raiz $\rightarrow C4$	$C4$	derrota	(1, 0)
5	raiz $\rightarrow C1$	(reuso) $C1$	vitoria	(2, 2)
6	raiz $\rightarrow C2$	(reuso) $C2$	vitoria	(2, 2)
7	raiz $\rightarrow C3$	(reuso) $C3$	derrota	(2, 1)
8	raiz $\rightarrow C1$	(reuso) $C1$	vitoria	(3, 3)
9	raiz $\rightarrow C2$	(reuso) $C2$	derrota	(3, 2)

6.5 Arvore parcial apos as 10 iteracoes

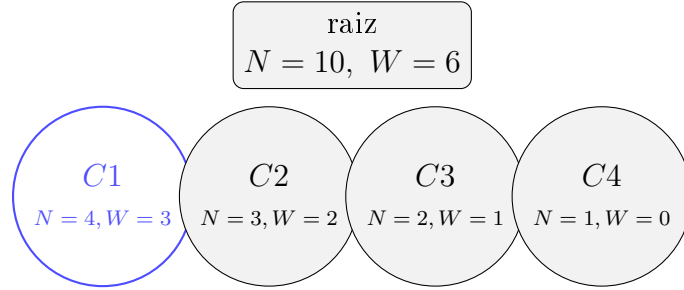


Figure 19: Arvore parcial do MCTS apos 10 iteracoes (azul = jogada mais visitada)

6.6 Tabela UCT apos 10 iteracoes

Para $N(\text{pai}) = 10$ e $C = 1.4$:

Jogada	N	W	$\frac{W}{N}$	$1.4\sqrt{\frac{\ln 10}{N}}$	UCT
C1	4	3	0.7500	1.0622	1.8122
C2	3	2	0.6667	1.2265	1.8932
C3	2	1	0.5000	1.5022	2.0022
C4	1	0	0.0000	2.1244	2.1244

Table 26: Calculo de UCT por filho da raiz

Leitura da tabela:

- maior *exploitation* (W/N): C1;
- maior UCT no instante: C4, por forte termo de exploracao (poucas visitas);
- decisao final de jogada no codigo: por maior numero de visitas, resultando em C1.

6.7 Sensibilidade ao parametro C (rollout semi-guloso)

C	Movimento mais estavel	Estabilidade	Diversidade media	Winrate medio
0.1	C1	57.5%	0.42	0.696
1.4	C1	75.0%	0.94	0.641
3.0	C1	72.5%	0.99	0.627

Table 27: Impacto do parametro de exploracao C

Interpretacao:

- $C = 0.1$: baixa exploracao, menor diversidade e tendencia mais gulosa;
- $C = 1.4$: melhor equilibrio entre explorar e consolidar decisao;
- $C = 3.0$: exploracao muito alta, aumentando diversidade, mas com queda de qualidade media.

6.8 Rollout aleatorio vs semi-guloso ($C=1.4$)

Politica de rollout	Movimento mais estavel	Estabilidade	Diversidade media	Winrate medio
Aleatorio	$C1$	70.0%	0.94	0.551
Semi-guloso	$C1$	75.0%	0.94	0.641

Table 28: Comparacao entre politicas de simulacao

Conclusao desta comparacao:

- semi-guloso foi melhor em qualidade (maior winrate medio);
- semi-guloso tambem foi ligeiramente mais estavel na escolha final;
- mantendo mesma diversidade media neste experimento.

6.9 Efeito do numero de iteracoes

Experimento adicional com $C = 1.4$:

Iteracoes	Rollout	Movimento estavel	Estabilidade	Winrate medio
10	semi-guloso	$C1$	65.0%	0.613
20	semi-guloso	$C1$	62.5%	0.611
40	semi-guloso	$C1$	75.0%	0.641
80	semi-guloso	$C1$	72.5%	0.631
120	semi-guloso	$C1$	82.5%	0.651
10	aleatorio	$C1$	62.5%	0.525
20	aleatorio	$C1$	55.0%	0.536
40	aleatorio	$C1$	70.0%	0.551
80	aleatorio	$C1$	67.5%	0.552
120	aleatorio	$C1$	72.5%	0.562

Table 29: Impacto do numero de iteracoes no MCTS

Em geral, mais iteracoes aumentam estabilidade da decisao e qualidade media, com ganho mais claro no rollout semi-guloso.

6.10 Comparacao final pedida na Q6

- (a) **Rollout aleatorio vs semi-guloso:** semi-guloso venceu em qualidade e estabilidade.
- (b) **Diferentes valores de C:** C baixo explora pouco; C alto explora bastante; $C = 1.4$ foi o melhor compromisso observado.
- (c) **Numero de iteracoes:** aumentar iteracoes melhorou consolidacao da jogada escolhida.
- (d) **Estabilidade das decisoes:** maior com semi-guloso e maior orcamento de simulacoes.

6.11 Resposta objetiva da jogada sugerida

No cenario principal (10 iteracoes, $C = 1.4$, rollout semi-guloso), a jogada escolhida pelo criterio de maior visita foi:

$C1$

A Codigos Python

A.1 Questao 1

```
1  """Questao 1 - BFS, DFS e IDS (implementacao autoral).
2
3  Regras atendidas:
4  - Sem bibliotecas externas.
5  - Sem bibliotecas prontas de grafos.
6  - Estruturas usadas: listas, dicionarios e funcoes simples.
7  """
8
9  from pathlib import Path
10
11
12  GRAPH_Q1 = {
13      "A": ["B", "C", "D"],
14      "B": ["E", "F"],
15      "C": ["G", "H"],
16      "D": ["I"],
17      "E": ["J"],
18      "F": ["K", "L"],
19      "G": ["M"],
20      "H": ["N", "O"],
21      "I": ["P"],
22      "J": [],
23      "K": ["Q"],
24      "L": [],
25      "M": ["R"],
26      "N": [],
27      "O": ["S"],
```

```

28     "P": [],
29     "Q": [],
30     "R": [],
31     "S": [],
32 }
33
34 START = "A"
35 GOAL = "S"
36
37
38 def reconstruct_path(parent, goal):
39     if goal not in parent:
40         return []
41     path = []
42     cur = goal
43     while cur is not None:
44         path.append(cur)
45         cur = parent[cur]
46     path.reverse()
47     return path
48
49
50 def bfs(graph, start, goal):
51     frontier = [start] # fila
52     visited = {start}
53     parent = {start: None}
54     expanded_order = []
55     trace = []
56     generated_count = 1
57
58     while frontier:
59         node = frontier.pop(0) # remove da frente da fila
60         expanded_order.append(node)
61
62         if node == goal:
63             trace.append((node, frontier.copy(), expanded_order.
64                           copy()))
65             break
66
67         for child in graph[node]:
68             if child not in visited:
69                 visited.add(child)
70                 parent[child] = node
71                 frontier.append(child)
72                 generated_count += 1
73
74         trace.append((node, frontier.copy(), expanded_order.copy()))
75
76     path = reconstruct_path(parent, goal)
77     return {

```

```

77         "name": "BFS",
78         "trace": trace,
79         "expanded_order": expanded_order,
80         "path": path,
81         "depth": len(path) - 1,
82         "cost": len(path) - 1,
83         "generated_count": generated_count,
84         "expanded_count": len(expanded_order),
85     }
86
87
88 def dfs(graph, start, goal):
89     frontier = [start] # pilha
90     visited = {start}
91     parent = {start: None}
92     expanded_order = []
93     trace = []
94     generated_count = 1
95
96     while frontier:
97         node = frontier.pop() # remove do topo
98         expanded_order.append(node)
99
100         if node == goal:
101             trace.append((node, frontier.copy(), expanded_order.
102                           copy()))
103             break
104
105         # Reverso para respeitar expansao da esquerda para direita.
106         for child in reversed(graph[node]):
107             if child not in visited:
108                 visited.add(child)
109                 parent[child] = node
110                 frontier.append(child)
111                 generated_count += 1
112
113         trace.append((node, frontier.copy(), expanded_order.copy())
114                     )
115
116     path = reconstruct_path(parent, goal)
117     return {
118         "name": "DFS",
119         "trace": trace,
120         "expanded_order": expanded_order,
121         "path": path,
122         "depth": len(path) - 1,
123         "cost": len(path) - 1,
124         "generated_count": generated_count,
125         "expanded_count": len(expanded_order),
126     }

```

```

126
127 def depth_limited_search(graph, start, goal, limit):
128     # pilha de (no, profundidade)
129     frontier = [(start, 0)]
130     parent = {start: None}
131     expanded_order = []
132     trace = []
133     generated_count = 1
134
135     # Como aqui o grafo eh uma arvore (sem ciclos), nao precisamos
136     # de estrutura de repeticao.
137     while frontier:
138         node, depth = frontier.pop()
139         expanded_order.append(node)
140
141         if node == goal:
142             trace.append((node, [n for n, _ in frontier],
143                           expanded_order.copy()))
144             path = reconstruct_path(parent, goal)
145             return {
146                 "found": True,
147                 "trace": trace,
148                 "expanded_order": expanded_order,
149                 "path": path,
150                 "generated_count": generated_count,
151                 "expanded_count": len(expanded_order),
152             }
153
154         if depth < limit:
155             for child in reversed(graph[node]):
156                 parent[child] = node
157                 frontier.append((child, depth + 1))
158                 generated_count += 1
159
160         trace.append((node, [n for n, _ in frontier],
161                       expanded_order.copy()))
162
163     return {
164         "found": False,
165         "trace": trace,
166         "expanded_order": expanded_order,
167         "path": [],
168         "generated_count": generated_count,
169         "expanded_count": len(expanded_order),
170     }
171
172 def ids(graph, start, goal, max_depth=30):
173     iterations = []
174     total_generated = 0
175     total_expanded = 0

```

```

174
175 for limit in range(max_depth + 1):
176     result = depth_limited_search(graph, start, goal, limit)
177     iterations.append(
178         {
179             "limit": limit,
180             "found": result["found"],
181             "trace": result["trace"],
182             "expanded_order": result["expanded_order"],
183             "path": result["path"],
184             "generated_count": result["generated_count"],
185             "expanded_count": result["expanded_count"],
186         }
187     )
188     total_generated += result["generated_count"]
189     total_expanded += result["expanded_count"]
190
191     if result["found"]:
192         return {
193             "name": "IDS",
194             "iterations": iterations,
195             "final_path": result["path"],
196             "final_depth": len(result["path"]) - 1,
197             "final_cost": len(result["path"]) - 1,
198             "total_generated": total_generated,
199             "total_expanded": total_expanded,
200         }
201
202     raise RuntimeError("Objetivo nao encontrado dentro do limite
203                         maximo.")
204
205 def apply_successor_change(base_graph, changes):
206     new_graph = {}
207     for node, children in base_graph.items():
208         new_graph[node] = children.copy()
209     for node, new_order in changes.items():
210         new_graph[node] = new_order.copy()
211     return new_graph
212
213
214 def format_trace_table(trace):
215     lines = ["Passo | No expandido | Fronteira apos expansao |
216             Visitados", "---|---|---|---"]
217     lines.append("0 | - | [A] | []")
218     for i, (node, frontier, visited) in enumerate(trace, start=1):
219         lines.append(f"{i} | {node} | {frontier} | {visited}")
220     return "\n".join(lines)
221
222 def build_report(title, graph):

```

```

223 bfs_result = bfs(graph, START, GOAL)
224 dfs_result = dfs(graph, START, GOAL)
225 ids_result = ids(graph, START, GOAL)
226
227 lines = []
228 lines.append(f"# {title}")
229 lines.append("")
230
231 lines.append("## BFS")
232 lines.append(format_trace_table(bfs_result["trace"]))
233 lines.append(
234     f"\nCaminho: {bfs_result['path']} | profundidade={
235         bfs_result['depth']} | "
236     f"custo={bfs_result['cost']} | gerados={bfs_result['
237         generated_count']} | "
238     f"expandidos={bfs_result['expanded_count']}"
239 )
240 lines.append("")
241
242 lines.append("## DFS")
243 lines.append(format_trace_table(dfs_result["trace"]))
244 lines.append(
245     f"\nCaminho: {dfs_result['path']} | profundidade={
246         dfs_result['depth']} | "
247     f"custo={dfs_result['cost']} | gerados={dfs_result['
248         generated_count']} | "
249     f"expandidos={dfs_result['expanded_count']}"
250 )
251 lines.append("")
252
253 lines.append("## IDS")
254 lines.append("Limite | Encontrou objetivo? | Gerados |
255     Expandidos | Ordem de expansao")
256 lines.append("---|---|---|---|---")
257 for it in ids_result["iterations"]:
258     found = "sim" if it["found"] else "nao"
259     lines.append(
260         f"{it['limit']} | {found} | {it['generated_count']} | "
261         f"{it['expanded_count']} | {it['expanded_order']}"
262     )
263 lines.append(
264     f"\nCaminho final: {ids_result['final_path']} |
265         profundidade={ids_result['final_depth']} | "
266     f"custo={ids_result['final_cost']} | gerados totais={
267         ids_result['total_generated']} | "
268     f"expandidos totais={ids_result['total_expanded']}"
269 )
270 lines.append("")
271
272 return "\n".join(lines)

```

```

267
268 def main():
269     original_report = build_report("Q1 - Resultado (ordem original)
        ", GRAPH_Q1)
270
271     changes = {
272         "C": ["H", "G"],
273         "F": ["L", "K"],
274     }
275     modified_graph = apply_successor_change(GRAPH_Q1, changes)
276     modified_report = build_report("Q1 - Resultado (ordem
        modificada em C e F)", modified_graph)
277
278     out_dir = Path(__file__).resolve().parent.parent / "results"
279     out_dir.mkdir(parents=True, exist_ok=True)
280     out_file = out_dir / "q1_resultados.md"
281     out_file.write_text(original_report + "\n\n" + modified_report
        + "\n", encoding="utf-8")
282
283     print(f"Arquivo gerado: {out_file}")
284     print("Implementacao usa apenas biblioteca padrao do Python.")
285
286
287 if __name__ == "__main__":
288     main()

```

A.2 Questao 2

```

1  """Questao 2 - Greedy Best-First Search e A* (implementacao autoral
2
3  Regras atendidas:
4  - Sem bibliotecas externas.
5  - Sem bibliotecas prontas de grafos.
6  - Apenas biblioteca padrao do Python.
7  """
8
9  from pathlib import Path
10 import heapq
11
12
13 GRAPH_Q2 = {
14     "A": [("B", 2), ("C", 4), ("D", 3)],
15     "B": [("E", 3), ("F", 5)],
16     "C": [("G", 4), ("H", 6)],
17     "D": [("I", 2)],
18     "E": [("J", 4)],
19     "F": [("K", 3), ("L", 5)],
20     "G": [("M", 6)],
21     "H": [("N", 3), ("O", 4)],

```



```

22     "I": [("P", 5)],
23     "J": [("Q", 4)],
24     "K": [("R", 3)],
25     "L": [],
26     "M": [("S", 2)],
27     "N": [],
28     "O": [("T", 5)],
29     "P": [],
30     "Q": [],
31     "R": [("T", 4)],
32     "S": [("T", 3)],
33     "T": [],
34 }
35
36 HEURISTIC_BASE = {
37     "A": 10,
38     "B": 8,
39     "C": 7,
40     "D": 9,
41     "E": 6,
42     "F": 5,
43     "G": 6,
44     "H": 4,
45     "I": 7,
46     "J": 5,
47     "K": 3,
48     "L": 6,
49     "M": 3,
50     "N": 4,
51     "O": 1,
52     "P": 8,
53     "Q": 4,
54     "R": 2,
55     "S": 1,
56     "T": 0,
57 }
58
59 START = "A"
60 GOAL = "T"
61
62
63 def reconstruct_path(parent, goal):
64     if goal not in parent:
65         return []
66     path = []
67     cur = goal
68     while cur is not None:
69         path.append(cur)
70         cur = parent[cur]
71     path.reverse()
72     return path

```

```

73
74
75 def path_cost(graph, path):
76     if not path:
77         return 0
78     cost = 0
79     for a, b in zip(path, path[1:]):
80         found = False
81         for child, w in graph[a]:
82             if child == b:
83                 cost += w
84                 found = True
85                 break
86         if not found:
87             raise ValueError(f"Aresta inexistente no caminho: {a
                                }->{b}")
88     return cost
89
90
91 def greedy_best_first(graph, heuristic, start, goal):
92     counter = 0
93     frontier = []
94     heapq.heappush(frontier, (heuristic[start], counter, start))
95     counter += 1
96
97     closed = set()
98     parent = {start: None}
99     expanded_order = []
100     trace = []
101     generated_count = 1
102
103     while frontier:
104         h_val, _, node = heapq.heappop(frontier)
105         if node in closed:
106             continue
107
108         closed.add(node)
109         expanded_order.append(node)
110
111         if node == goal:
112             snapshot = sorted([(h, n) for h, _, n in frontier], key
                                =lambda x: (x[0], x[1]))
113             trace.append((node, h_val, snapshot, expanded_order.
                            copy()))
114             break
115
116         for child, _ in graph[node]:
117             if child not in closed:
118                 if child not in parent:
119                     parent[child] = node

```

```

120         heapq.heappush(frontier, (heuristic[child], counter
121                                   , child))
122         counter += 1
123         generated_count += 1
124
125     snapshot = sorted([(h, n) for h, _, n in frontier], key=
126                       lambda x: (x[0], x[1]))
127     trace.append((node, h_val, snapshot, expanded_order.copy()))
128
129     path = reconstruct_path(parent, goal)
130     return {
131         "name": "Greedy Best-First",
132         "trace": trace,
133         "expanded_order": expanded_order,
134         "path": path,
135         "depth": len(path) - 1,
136         "cost": path_cost(graph, path),
137         "generated_count": generated_count,
138         "expanded_count": len(expanded_order),
139     }
140
141 def a_star(graph, heuristic, start, goal):
142     counter = 0
143     frontier = []
144     # item: (f, g, counter, node)
145     heapq.heappush(frontier, (heuristic[start], 0, counter, start))
146     counter += 1
147
148     best_g = {start: 0}
149     parent = {start: None}
150     closed = set()
151     expanded_order = []
152     trace = []
153     generated_count = 1
154
155     while frontier:
156         f_val, g_val, _, node = heapq.heappop(frontier)
157         if node in closed:
158             continue
159         if g_val != best_g[node]:
160             continue
161
162         closed.add(node)
163         expanded_order.append(node)
164
165         if node == goal:
166             snapshot = sorted([(f, g, n) for f, g, _, n in frontier
167                               ], key=lambda x: (x[0], x[1], x[2]))

```

```

166         trace.append((node, g_val, heuristic[node], f_val,
167                        snapshot, expanded_order.copy()))
168         break
169     for child, w in graph[node]:
170         ng = g_val + w
171         if child not in best_g or ng < best_g[child]:
172             best_g[child] = ng
173             parent[child] = node
174             heapq.heappush(frontier, (ng + heuristic[child], ng
175                                     , counter, child))
176             counter += 1
177             generated_count += 1
178
179     snapshot = sorted([(f, g, n) for f, g, _, n in frontier],
180                       key=lambda x: (x[0], x[1], x[2]))
181     trace.append((node, g_val, heuristic[node], f_val, snapshot
182                 , expanded_order.copy()))
183
184     path = reconstruct_path(parent, goal)
185     return {
186         "name": "A*",
187         "trace": trace,
188         "expanded_order": expanded_order,
189         "path": path,
190         "depth": len(path) - 1,
191         "cost": path_cost(graph, path),
192         "generated_count": generated_count,
193         "expanded_count": len(expanded_order),
194     }
195
196 def run_set(title, graph, heuristic):
197     greedy = greedy_best_first(graph, heuristic, START, GOAL)
198     astar = a_star(graph, heuristic, START, GOAL)
199     return {"title": title, "greedy": greedy, "astar": astar, "
200            heuristic": heuristic}
201
202 def trace_table_greedy(trace):
203     lines = [
204         "Passo | No | h(n) | Fronteira ordenada por h | Visitados",
205         "----|---|---|---|---",
206     ]
207     for i, (node, h_val, frontier, visited) in enumerate(trace,
208                                                            start=1):
209         frontier_txt = "[" + ", ".join([f"{n}:{h}" for h, n in
210                                         frontier]) + "]"
211         lines.append(f"{i} | {node} | {h_val} | {frontier_txt} | {
212                       visited}")
213     return "\n".join(lines)

```

```

209
210
211 def trace_table_astar(trace, heuristic):
212     lines = [
213         "Passo | No | g(n) | h(n) | f(n) | Fronteira ordenada por f
          | Visitados",
214         "----|----|----|----|----|----|----",
215     ]
216     for i, (node, g_val, h_val, f_val, frontier, visited) in
        enumerate(trace, start=1):
217         frontier_txt = "[" + ", ".join([f"{n}:(g={g},h={heuristic.
          get(n,'?')},f={f})" for f, g, n in frontier]) + "]"
218         lines.append(f"{i} | {node} | {g_val} | {h_val} | {f_val} |
          {frontier_txt} | {visited}")
219     return "\n".join(lines)
220
221
222 def build_report(run_original, run_modified, modified_changes):
223     lines = []
224     for run in [run_original, run_modified]:
225         lines.append(f"# {run['title']}")
226         lines.append("")
227         lines.append("## Greedy")
228         lines.append(trace_table_greedy(run["greedy"]["trace"]))
229         lines.append(
230             f"\nCaminho: {run['greedy']['path']} | profundidade={
          run['greedy']['depth']} | "
231             f"custo={run['greedy']['cost']} | gerados={run['greedy
          ']['generated_count']} | "
232             f"expandidos={run['greedy']['expanded_count']}"
233         )
234         lines.append("")
235         lines.append("## A*")
236         lines.append(trace_table_astar(run["astar"]["trace"], run["
          heuristic"]))
237         lines.append(
238             f"\nCaminho: {run['astar']['path']} | profundidade={run
          ['astar']['depth']} | "
239             f"custo={run['astar']['cost']} | gerados={run['astar
          ']['generated_count']} | "
240             f"expandidos={run['astar']['expanded_count']}"
241         )
242         lines.append("")
243
244     lines.append("# Heurísticas modificadas")
245     lines.append("")
246     for n, (old, new) in modified_changes.items():
247         lines.append(f"- {n}: {old} -> {new}")
248     lines.append("")
249     return "\n".join(lines)
250

```

```

251
252 def main():
253     run_original = run_set("Q2 - Resultado (heurística original)",
254                             GRAPH_Q2, HEURISTIC_BASE)
255
256     heuristic_modified = dict(HEURISTIC_BASE)
257     heuristic_modified["B"] = 2
258     heuristic_modified["H"] = 8
259     heuristic_modified["G"] = 10
260     run_modified = run_set("Q2 - Resultado (heurística modificada
261                             em 3 nos)", GRAPH_Q2, heuristic_modified)
262
263     modified_changes = {
264         "B": (HEURISTIC_BASE["B"], heuristic_modified["B"]),
265         "H": (HEURISTIC_BASE["H"], heuristic_modified["H"]),
266         "G": (HEURISTIC_BASE["G"], heuristic_modified["G"]),
267     }
268
269     report = build_report(run_original, run_modified,
270                           modified_changes)
271     out_dir = Path(__file__).resolve().parent.parent / "results"
272     out_dir.mkdir(parents=True, exist_ok=True)
273     out_file = out_dir / "q2_resultados.md"
274     out_file.write_text(report + "\n", encoding="utf-8")
275
276     print(f"Arquivo gerado: {out_file}")
277     print("Heurísticas alteradas: B, H, G")
278
279 if __name__ == "__main__":
280     main()

```

A.3 Questao 3

```

1  """Questao 3 - Busca Local (8-Rainhas), implementacao autoral.
2
3  Regras atendidas:
4  - Sem bibliotecas externas.
5  - Sem bibliotecas prontas de IA/grafos.
6  - Apenas biblioteca padrao do Python.
7  """
8
9  from __future__ import annotations
10
11  import math
12  import random
13  from pathlib import Path
14
15  N = 8
16  INITIAL_STATE = (1, 1, 1, 1, 1, 1, 1, 1)

```

```

17
18
19 def state_to_str(state: tuple[int, ...]) -> str:
20     return "[" + ",".join(str(x) for x in state) + "]"
21
22
23 def count_conflicts(state: tuple[int, ...]) -> int:
24     """Numero de pares em conflito (mesma linha ou diagonal)."""
25     conflicts = 0
26     for c1 in range(N):
27         r1 = state[c1]
28         for c2 in range(c1 + 1, N):
29             r2 = state[c2]
30             same_row = r1 == r2
31             same_diag = abs(r1 - r2) == abs(c1 - c2)
32             if same_row or same_diag:
33                 conflicts += 1
34     return conflicts
35
36
37 def neighbors(state: tuple[int, ...]) -> list[tuple[int, ...]]:
38     """Todos os vizinhos: move 1 rainha para outra linha na mesma
39         coluna."""
40     out = []
41     for c in range(N):
42         cur_r = state[c]
43         for r in range(1, N + 1):
44             if r == cur_r:
45                 continue
46             s = list(state)
47             s[c] = r
48             out.append(tuple(s))
49     return out
50
51 def sorted_neighbors_with_h(state: tuple[int, ...]) -> list[tuple[
52     int, tuple[int, ...]]]:
53     """Ordena por h crescente e desempata lexicograficamente."""
54     scored = [(count_conflicts(nb), nb) for nb in neighbors(state)]
55     scored.sort(key=lambda x: (x[0], x[1]))
56     return scored
57
58 def hill_climbing(
59     start: tuple[int, ...],
60     topk: int = 5,
61 ) -> dict:
62     """Steepest-ascent (na pratica, steepest-descent em h)."""
63     cur = start
64     cur_h = count_conflicts(cur)
65     trace = []

```

```

66     iter_idx = 0
67
68     while True:
69         ranked = sorted_neighbors_with_h(cur)
70         best_h, best_state = ranked[0]
71         top = ranked[:topk]
72
73         if best_h < cur_h:
74             trace.append(
75                 {
76                     "iter": iter_idx,
77                     "state": cur,
78                     "h": cur_h,
79                     "top": top,
80                     "chosen": best_state,
81                     "chosen_h": best_h,
82                     "reason": f"melhor vizinho (h={best_h})",
83                 }
84             )
85             cur = best_state
86             cur_h = best_h
87             iter_idx += 1
88             continue
89
90         # Nao ha melhoria: parada.
91         equal_exists = any(hv == cur_h for hv, _ in ranked)
92         if cur_h == 0:
93             trap = "solucao global"
94             reason = "h=0 (solucao valida)"
95         elif equal_exists:
96             trap = "plato"
97             reason = "nao existe vizinho com h menor"
98         else:
99             trap = "minimo local"
100            reason = "todos os vizinhos sao piores"
101
102            trace.append(
103                {
104                    "iter": iter_idx,
105                    "state": cur,
106                    "h": cur_h,
107                    "top": top,
108                    "chosen": None,
109                    "chosen_h": None,
110                    "reason": f"parou: {reason}",
111                }
112            )
113            break
114
115     return {
116         "start": start,

```



```

117         "final": cur,
118         "final_h": cur_h,
119         "steps": max(0, len(trace) - 1),
120         "trace": trace,
121         "trap": trap,
122         "success": cur_h == 0,
123     }
124
125
126 def random_state(rng: random.Random) -> tuple[int, ...]:
127     return tuple(rng.randint(1, N) for _ in range(N))
128
129
130 def random_restart_hc(
131     runs: int = 20,
132     seed: int = 2026,
133 ) -> list[dict]:
134     rng = random.Random(seed)
135     out = []
136     for i in range(1, runs + 1):
137         s0 = random_state(rng)
138         r = hill_climbing(s0)
139         out.append(
140             {
141                 "run": i,
142                 "start": s0,
143                 "steps": r["steps"],
144                 "final_h": r["final_h"],
145                 "success": r["success"],
146                 "final": r["final"],
147             }
148         )
149     return out
150
151
152 def random_neighbor(state: tuple[int, ...], rng: random.Random) ->
tuple[int, ...]:
153     c = rng.randint(0, N - 1)
154     cur_r = state[c]
155     r = rng.randint(1, N)
156     while r == cur_r:
157         r = rng.randint(1, N)
158     s = list(state)
159     s[c] = r
160     return tuple(s)
161
162
163 def simulated_annealing(
164     start: tuple[int, ...],
165     rng: random.Random,
166     t0: float = 25.0,

```

```

167     alpha: float = 0.97,
168     tmin: float = 0.001,
169     max_steps: int = 3000,
170 ) -> dict:
171     cur = start
172     cur_h = count_conflicts(cur)
173     best = cur
174     best_h = cur_h
175     t = t0
176     accepted_worse = []
177
178     step = 0
179     while step < max_steps and t > tmin and best_h > 0:
180         nxt = random_neighbor(cur, rng)
181         nxt_h = count_conflicts(nxt)
182         delta = nxt_h - cur_h
183
184         accepted = False
185         prob = None
186         draw = None
187
188         if delta <= 0:
189             accepted = True
190         else:
191             prob = math.exp(-delta / t)
192             draw = rng.random()
193             if draw < prob:
194                 accepted = True
195                 if len(accepted_worse) < 8:
196                     accepted_worse.append(
197                         {
198                             "step": step,
199                             "from_h": cur_h,
200                             "to_h": nxt_h,
201                             "delta": delta,
202                             "temp": t,
203                             "prob": prob,
204                             "draw": draw,
205                         }
206                     )
207
208             if accepted:
209                 cur = nxt
210                 cur_h = nxt_h
211                 if cur_h < best_h:
212                     best = cur
213                     best_h = cur_h
214
215             t *= alpha
216             step += 1
217

```

```

218     return {
219         "start": start,
220         "final": cur,
221         "final_h": cur_h,
222         "best": best,
223         "best_h": best_h,
224         "steps": step,
225         "success": best_h == 0,
226         "accepted_worse": accepted_worse,
227         "t0": t0,
228         "alpha": alpha,
229         "tmin": tmin,
230     }
231
232
233 def run_sa_batch(runs: int = 20, seed: int = 9090) -> list[dict]:
234     rng = random.Random(seed)
235     out = []
236     for i in range(1, runs + 1):
237         s0 = random_state(rng)
238         r = simulated_annealing(s0, rng)
239         out.append(
240             {
241                 "run": i,
242                 "start": s0,
243                 "steps": r["steps"],
244                 "best_h": r["best_h"],
245                 "success": r["success"],
246             }
247         )
248     return out
249
250
251 def mean(values: list[float]) -> float:
252     if not values:
253         return 0.0
254     return sum(values) / len(values)
255
256
257 def build_markdown(
258     hc_manual: dict,
259     rr_runs: list[dict],
260     sa_demo: dict,
261     sa_runs: list[dict],
262 ) -> str:
263     lines = []
264     lines.append("# Q3 - Resultados (8-Rainhas)")
265     lines.append("")
266
267     lines.append("## 1) Hill-Climbing manual (inicio
        [1,1,1,1,1,1,1,1])")

```

```

268 lines.append("Iter | Estado atual | h | Escolha")
269 lines.append("---|---|---|---")
270 for row in hc_manual["trace"]:
271     st = state_to_str(row["state"])
272     if row["chosen"] is None:
273         choice = row["reason"]
274     else:
275         choice = f"{state_to_str(row['chosen'])} (h={row['chosen_h']}) - {row['reason']}"
276     lines.append(f"{row['iter']} | {st} | {row['h']} | {choice}")
277 lines.append("")
278 lines.append(
279     f"Final: {state_to_str(hc_manual['final'])}, h={hc_manual['final_h']}, "
280     f"trap={hc_manual['trap']}, sucesso={hc_manual['success']}"
281 )
282 lines.append("")
283
284 lines.append("### Top-5 vizinhos por iteracao")
285 lines.append("Iter | Vizinho | h")
286 lines.append("---|---|---")
287 for row in hc_manual["trace"]:
288     for hv, st in row["top"]:
289         lines.append(f"{row['iter']} | {state_to_str(st)} | {hv}")
290 lines.append("")
291
292 lines.append("## 2) Random Restart Hill-Climbing (20 execucoes)")
293 lines.append("Run | Estado inicial | Passos | h final | Solucao")
294 lines.append("---|---|---:|---:|---")
295 for r in rr_runs:
296     yn = "sim" if r["success"] else "nao"
297     lines.append(
298         f"{r['run']} | {state_to_str(r['start'])} | {r['steps']}"
299         f"| {r['final_h']} | {yn}"
300     )
301 rr_success = sum(1 for r in rr_runs if r["success"])
302 lines.append("")
303 lines.append(
304     f"Taxa de sucesso RR-HC: {rr_success}/20 = {100.0*rr_success/20:.1f}% | "
305     f"media h final={mean([r['final_h'] for r in rr_runs]):.2f}"
306     f"| "
307     f"media passos={mean([r['steps'] for r in rr_runs]):.2f}"
308 )
309 lines.append("")

```

```

309 lines.append("## 3) Simulated Annealing (1 execucao detalhada +
    lote de 20)")
310 lines.append(
311     f"Demo SA: inicio={state_to_str(sa_demo['start'])}, "
312     f"final={state_to_str(sa_demo['final'])}, best_h={sa_demo['best_h']}, "
313     f"sucesso={sa_demo['success']}, passos={sa_demo['steps']}"
314 )
315 lines.append(
316     f"Parametros: T0={sa_demo['t0']}, alpha={sa_demo['alpha']},
        Tmin={sa_demo['tmin']}"
317 )
318 lines.append("")
319 lines.append("Movimentos piores aceitos (demo):")
320 lines.append("Step | h atual -> h novo | Delta | T | exp(-Delta
    /T) | sorteio")
321 lines.append("----|---|---:|---:|---:|---:")
322 for m in sa_demo["accepted_worse"]:
323     lines.append(
324         f"{m['step']} | {m['from_h']} -> {m['to_h']} | +{m['delta']} | "
325         f"{m['temp']:.4f} | {m['prob']:.4f} | {m['draw']:.4f}"
326     )
327 lines.append("")
328
329 sa_success = sum(1 for r in sa_runs if r["success"])
330 lines.append("Lote SA (20 execucoes):")
331 lines.append("Run | Estado inicial | Passos | best h | Solucao"
    )
332 lines.append("---|---|---:|---:|---")
333 for r in sa_runs:
334     yn = "sim" if r["success"] else "nao"
335     lines.append(
336         f"{r['run']} | {state_to_str(r['start'])} | {r['steps']} | {r['best_h']} | {yn}"
337     )
338 lines.append("")
339 lines.append(
340     f"Taxa de sucesso SA: {sa_success}/20 = {100.0*sa_success/20:.1f}% | "
341     f"media best h={mean([r['best_h'] for r in sa_runs]):.2f} | "
342     f"media passos={mean([r['steps'] for r in sa_runs]):.2f}"
343 )
344 lines.append("")
345
346 return "\n".join(lines).rstrip() + "\n"
347
348
349 def build_hc_all_neighbors_tex(hc_manual: dict) -> str:

```

```

350     """Tabela LaTeX com todos os vizinhos avaliados em cada
        iteracao do HC."""
351     lines = []
352     lines.append("% Arquivo gerado automaticamente por
        q3_busca_local.py")
353     lines.append(r"\begin{group}")
354     lines.append(r"\scriptsize")
355     lines.append(r"\setlength{\tabcolsep}{3.5pt}")
356     lines.append(r"\renewcommand{\arraystretch}{1.1}")
357     lines.append(r"\begin{longtable}[c p{3.8cm} p{3.8cm} c]")
358     lines.append(r"\toprule")
359     lines.append(r"Iteracao & Estado atual & Vizinho avaliado & $h(
        s)$ \\\")
360     lines.append(r"\midrule")
361
362     for row in hc_manual["trace"]:
363         ranked = sorted_neighbors_with_h(row["state"])
364         for idx, (hv, nb) in enumerate(ranked):
365             iter_cell = str(row["iter"]) if idx == 0 else ""
366             state_cell = state_to_str(row["state"]) if idx == 0
                 else ""
367             lines.append(
368                 f"{iter_cell} & {state_cell} & {state_to_str(nb)} &
                    {hv} \\\\"
369             )
370
371     lines.append(r"\bottomrule")
372     lines.append(r"\end{longtable}")
373     lines.append(r"\endgroup")
374     return "\n".join(lines).rstrip() + "\n"
375
376
377 def main() -> None:
378     hc_manual = hill_climbing(INITIAL_STATE)
379     rr_runs = random_restart_hc(runs=20, seed=2026)
380
381     rng_demo = random.Random(707)
382     sa_demo_start = random_state(rng_demo)
383     sa_demo = simulated_annealing(sa_demo_start, rng_demo)
384     sa_runs = run_sa_batch(runs=20, seed=9090)
385
386     md = build_markdown(hc_manual, rr_runs, sa_demo, sa_runs)
387     out_dir = Path(__file__).resolve().parent.parent / "results"
388     out_dir.mkdir(parents=True, exist_ok=True)
389     out_file = out_dir / "q3_resultados.md"
390     out_file.write_text(md, encoding="utf-8")
391     all_neighbors_tex = build_hc_all_neighbors_tex(hc_manual)
392     all_neighbors_file = out_dir / "q3_hc_todos_vizinhos.tex"
393     all_neighbors_file.write_text(all_neighbors_tex, encoding="utf
        -8")
394

```

```

395     print("Q3 concluida.")
396     print(f"Arquivo gerado: {out_file}")
397     print(f"Tabela LaTeX (todos os vizinhos): {all_neighbors_file}"
398           )
399     print(f"HC manual: final_h={hc_manual['final_h']} | trap={
400           hc_manual['trap']}")
401     print(
402         "RR-HC sucesso: "
403         f"{sum(1 for r in rr_runs if r['success'])}/{len(rr_runs)}"
404     )
405     print(
406         "SA sucesso: "
407         f"{sum(1 for r in sa_runs if r['success'])}/{len(sa_runs)}"
408     )
409 if __name__ == "__main__":
410     main()

```

A.4 Questao 4

```

1  """Questao 4 - CSP e otimizacoes de busca.
2
3  Implementacao autoral com:
4  - Backtracking (ordem fixa)
5  - Backtracking + MRV
6  - Backtracking + MRV + Degree
7  - Forward Checking (com MRV + Degree)
8  - Backjumping (CBJ simplificado)
9
10 Sem bibliotecas externas.
11 """
12
13 from __future__ import annotations
14
15 from dataclasses import dataclass, field
16 from pathlib import Path
17
18
19 TURNOS = ["T1", "T2", "T3", "T4", "T5", "T6"]
20 MEDICOS = ["A", "B", "C", "D"]
21
22 # Restricoes modeladas (coerentes com a OCR recuperada)
23 UNARY_FORBIDDEN = {
24     "T2": {"A"},
25     "T5": {"A"},
26 }
27 NEQ_EDGES = [("T1", "T2"), ("T2", "T3"), ("T3", "T4"), ("T4", "T5"),
28              , ("T5", "T6")]

```

```

28 SPECIAL_PAIR_C = ("T2", "T5") # C nao pode estar nos dois
    simultaneamente
29 B_COVER_PAIR = ("T3", "T4") # B deve estar em pelo menos um deles
30 MAX_D = 2 # D no maximo 2 turnos
31
32
33 def state_to_str(assignment: dict[str, str]) -> str:
34     parts = [f"{t}={assignment[t]}" for t in TURNOS if t in
        assignment]
35     return "{" + ", ".join(parts) + "}"
36
37
38 def base_domains() -> dict[str, set[str]]:
39     d: dict[str, set[str]] = {t: set(MEDICOS) for t in TURNOS}
40     for t, forbidden in UNARY_FORBIDDEN.items():
41         d[t] -= forbidden
42     return d
43
44
45 def neighbors_for_degree(var: str) -> list[str]:
46     neigh = set()
47     for a, b in NEQ_EDGES:
48         if a == var:
49             neigh.add(b)
50         elif b == var:
51             neigh.add(a)
52     if var in SPECIAL_PAIR_C:
53         other = SPECIAL_PAIR_C[1] if var == SPECIAL_PAIR_C[0] else
            SPECIAL_PAIR_C[0]
54         neigh.add(other)
55     if var in B_COVER_PAIR:
56         other = B_COVER_PAIR[1] if var == B_COVER_PAIR[0] else
            B_COVER_PAIR[0]
57         neigh.add(other)
58     return sorted(neigh)
59
60
61 DEGREE_NEIGHBORS = {t: neighbors_for_degree(t) for t in TURNOS}
62
63
64 def d_count(assignment: dict[str, str]) -> int:
65     return sum(1 for v in assignment.values() if v == "D")
66
67
68 def immediate_conflicts(var: str, value: str, assignment: dict[str,
    str]) -> set[str]:
69     """Retorna variaveis ja atribuidas que conflitam com var=value.
        """
70     conflicts: set[str] = set()
71
72     # Restricoes unarias

```



```

73     if var in UNARY_FORBIDDEN and value in UNARY_FORBIDDEN[var]:
74         conflicts.add(var)
75
76     # Restricoes binarias de consecutividade
77     for a, b in NEQ_EDGES:
78         if a == var and b in assignment and assignment[b] == value:
79             conflicts.add(b)
80         if b == var and a in assignment and assignment[a] == value:
81             conflicts.add(a)
82
83     # Restricao C em T2 e T5
84     if var in SPECIAL_PAIR_C and value == "C":
85         other = SPECIAL_PAIR_C[1] if var == SPECIAL_PAIR_C[0] else
            SPECIAL_PAIR_C[0]
86         if other in assignment and assignment[other] == "C":
87             conflicts.add(other)
88
89     # Restricao global de D
90     if value == "D":
91         d_vars = [k for k, v in assignment.items() if v == "D"]
92         if len(d_vars) >= MAX_D:
93             conflicts.update(d_vars)
94
95     # Restricao B em pelo menos um entre T3 e T4 (checagem parcial
            local)
96     if var in B_COVER_PAIR and value != "B":
97         other = B_COVER_PAIR[1] if var == B_COVER_PAIR[0] else
            B_COVER_PAIR[0]
98         if other in assignment and assignment[other] != "B":
99             conflicts.add(other)
100
101     return conflicts
102
103
104 def is_consistent(assignment: dict[str, str]) -> bool:
105     # Unarias
106     for t, forbidden in UNARY_FORBIDDEN.items():
107         if t in assignment and assignment[t] in forbidden:
108             return False
109
110     # Consecutividade
111     for a, b in NEQ_EDGES:
112         if a in assignment and b in assignment and assignment[a] ==
            assignment[b]:
113             return False
114
115     # Par C
116     a, b = SPECIAL_PAIR_C
117     if a in assignment and b in assignment:
118         if assignment[a] == "C" and assignment[b] == "C":
119             return False

```

```

120
121     # B em T3 ou T4
122     t3, t4 = B_COVER_PAIR
123     if t3 in assignment and t4 in assignment:
124         if assignment[t3] != "B" and assignment[t4] != "B":
125             return False
126
127     # D no maximo dois
128     if d_count(assignment) > MAX_D:
129         return False
130
131     return True
132
133
134 def select_fixed(unassigned: list[str], _: dict[str, set[str]], __:
135     dict[str, str]) -> str:
136     return unassigned[0]
137
138 def select_mrv(unassigned: list[str], domains: dict[str, set[str]],
139     _: dict[str, str]) -> str:
140     return min(unassigned, key=lambda v: (len(domains[v]), TURNOS.
141         index(v)))
142
143
144 def select_mrv_degree(unassigned: list[str], domains: dict[str, set
145     [str]], assignment: dict[str, str]) -> str:
146     def key(v: str):
147         mrv = len(domains[v])
148         deg = sum(1 for n in DEGREE_NEIGHBORS[v] if n not in
149             assignment)
150         return (mrv, -deg, TURNOS.index(v))
151
152     return min(unassigned, key=key)
153
154
155 def forward_prune(
156     var: str,
157     value: str,
158     assignment: dict[str, str],
159     domains: dict[str, set[str]],
160 ) -> tuple[bool, dict[str, set[str]]]:
161     """Aplica Forward Checking e retorna (ok, novos_dominios)."""
162     new_domains = {k: set(v) for k, v in domains.items()}
163     new_domains[var] = {value}
164
165     # Consecutividade: vizinhos consecutivos nao podem repetir
166     valor
167     for a, b in NEQ_EDGES:
168         if a == var and b not in assignment and value in
169             new_domains[b]:

```

```

164         new_domains[b].remove(value)
165     if b == var and a not in assignment and value in
        new_domains[a]:
166         new_domains[a].remove(value)
167
168     # Restricao C em T2 e T5
169     if var in SPECIAL_PAIR_C and value == "C":
170         other = SPECIAL_PAIR_C[1] if var == SPECIAL_PAIR_C[0] else
            SPECIAL_PAIR_C[0]
171         if other not in assignment and "C" in new_domains[other]:
172             new_domains[other].remove("C")
173
174     # Restricao global de D
175     if value == "D":
176         d_now = d_count(assignment) + 1
177         if d_now >= MAX_D:
178             for u in TURNOS:
179                 if u not in assignment and u != var and "D" in
                    new_domains[u]:
180                     new_domains[u].remove("D")
181
182     # Restricao B em T3 ou T4
183     t3, t4 = B_COVER_PAIR
184     if var == t3 and value != "B" and t4 not in assignment:
185         new_domains[t4].intersection_update({"B"})
186     if var == t4 and value != "B" and t3 not in assignment:
187         new_domains[t3].intersection_update({"B"})
188
189     # Nenhum dominio pode ficar vazio
190     for u in TURNOS:
191         if u not in assignment and u != var and len(new_domains[u])
            == 0:
192             return False, new_domains
193
194     return True, new_domains
195
196
197 @dataclass
198 class Stats:
199     name: str
200     states_explored: int = 0
201     consistency_checks: int = 0
202     backtracks: int = 0
203     max_depth: int = 0
204     jumps: int = 0
205     trace: list[dict] = field(default_factory=list)
206
207
208 def solve_backtracking_like(
209     name: str,
210     select_var_fn,

```

```

211     use_fc: bool = False,
212     use_lcv: bool = False,
213 ) -> tuple[bool, dict[str, str], Stats]:
214     stats = Stats(name=name)
215     domains0 = base_domains()
216     assignment: dict[str, str] = {}
217
218     def rec(current_domains: dict[str, set[str]]) -> bool:
219         stats.max_depth = max(stats.max_depth, len(assignment))
220         if len(assignment) == len(TURNOS):
221             return True
222
223         unassigned = [v for v in TURNOS if v not in assignment]
224         var = select_var_fn(unassigned, current_domains, assignment
225                             )
226
227         if use_lcv:
228             candidate_values = []
229             for vv in MEDICOS:
230                 if vv not in current_domains[var]:
231                     continue
232                 conf = len(immediate_conflicts(var, vv, assignment)
233                             )
234                 candidate_values.append((conf, vv))
235                 candidate_values.sort(key=lambda x: (x[0], MEDICOS.
236                                     index(x[1])))
237             values_to_try = [v for _c, v in candidate_values]
238         else:
239             values_to_try = [v for v in MEDICOS if v in
240                             current_domains[var]]
241
242         for value in values_to_try:
243             stats.states_explored += 1
244             conflicts = immediate_conflicts(var, value, assignment)
245             if conflicts:
246                 stats.trace.append(
247                     {
248                         "var": var,
249                         "value": value,
250                         "status": "conflito",
251                         "state": state_to_str(**assignment, var:
252                                             value)},
253                     )
254                 continue
255
256             assignment[var] = value
257             stats.consistency_checks += 1
258             if not is_consistent(assignment):
259                 stats.trace.append(
260                     {

```

```

257         "var": var,
258         "value": value,
259         "status": "conflito",
260         "state": state_to_str(assignment),
261     }
262 )
263 assignment.pop(var, None)
264 continue
265
266 next_domains = current_domains
267 if use_fc:
268     ok, pruned = forward_prune(var, value, assignment,
269                               current_domains)
270     if not ok:
271         stats.trace.append(
272             {
273                 "var": var,
274                 "value": value,
275                 "status": "conflito-fc",
276                 "state": state_to_str(assignment),
277             }
278         )
279         assignment.pop(var, None)
280         continue
281     next_domains = pruned
282
283 stats.trace.append(
284     {
285         "var": var,
286         "value": value,
287         "status": "segue",
288         "state": state_to_str(assignment),
289     }
290 )
291
292 if rec(next_domains):
293     return True
294
295 stats.backtracks += 1
296 assignment.pop(var, None)
297
298 return False
299
300 ok = rec(domains0)
301 return ok, assignment.copy(), stats
302
303 def solve_backjumping() -> tuple[bool, dict[str, str], Stats]:
304     """CBJ simplificado com ordem fixa T1..T6."""
305     stats = Stats(name="Backjumping")
306     domains = base_domains()

```

```

307 assignment: dict[str, str] = {}
308
309 def rec(index: int) -> tuple[bool, set[str]]:
310     stats.max_depth = max(stats.max_depth, len(assignment))
311     if index == len(TURNOS):
312         return True, set()
313
314     var = TURNOS[index]
315     conflict_set: set[str] = set()
316
317     for value in MEDICOS:
318         if value not in domains[var]:
319             continue
320         stats.states_explored += 1
321         local_conflicts = immediate_conflicts(var, value,
322                                             assignment)
323         if local_conflicts:
324             conflict_set.update(local_conflicts)
325             stats.trace.append(
326                 {
327                     "var": var,
328                     "value": value,
329                     "status": "conflicto",
330                     "state": state_to_str(**assignment, var:
331                                         value)},
332                 )
333             continue
334
335         assignment[var] = value
336         stats.consistency_checks += 1
337         if not is_consistent(assignment):
338             stats.trace.append(
339                 {
340                     "var": var,
341                     "value": value,
342                     "status": "conflicto",
343                     "state": state_to_str(assignment),
344                 }
345             )
346         assignment.pop(var, None)
347         conflict_set.add(var)
348         continue
349
350     stats.trace.append(
351         {
352             "var": var,
353             "value": value,
354             "status": "segue",
355             "state": state_to_str(assignment),
356         }

```

```

356         )
357         ok, child_conflicts = rec(index + 1)
358         if ok:
359             return True, set()
360
361         assignment.pop(var, None)
362         stats.backtracks += 1
363
364         # Se var nao esta no conjunto de conflito retornado,
365         # pode saltar acima
366         if var not in child_conflicts and child_conflicts:
367             stats.jumps += 1
368             return False, child_conflicts
369
370         conflict_set.update(child_conflicts - {var})
371
372         if not conflict_set:
373             conflict_set.add(var)
374             return False, conflict_set
375
376         ok, _ = rec(0)
377         return ok, assignment.copy(), stats
378
379 def first_n_rows(trace: list[dict], n: int = 10) -> list[dict]:
380     return trace[:n]
381
382
383 def metrics_table(rows: list[tuple[str, Stats, dict[str, str]]]) ->
384     str:
385     out = []
386     out.append("Algoritmo | Solucao | Estados explorados |
387         Backtracks | Prof max | Saltos BJ")
388     out.append("---|---|---:|---:|---:|---:")
389     for name, stats, sol in rows:
390         sol_s = "(" + ",".join(sol.get(t, "-") for t in TURNOS) + "
391             )" if sol else "-"
392         out.append(
393             f"{name} | {sol_s} | {stats.states_explored} | {stats.
394                 backtracks} | {stats.max_depth} | {stats.jumps}"
395         )
396     return "\n".join(out)
397
398
399 def build_report(
400     runs: list[tuple[str, bool, dict[str, str], Stats]],
401     trace_bt: list[dict],
402     trace_fc: list[dict],
403     trace_bj: list[dict],
404 ) -> str:
405     lines = []

```

```

402 lines.append("# Q4 - Resultados CSP")
403 lines.append("")
404 lines.append("## Modelagem usada")
405 lines.append("- Variaveis: T1..T6")
406 lines.append("- Dominio: {A,B,C,D}")
407 lines.append("- Restricoes: consecutividade (!=), T2!=A, T5!=A,
      B em T3 ou T4, C nao simultaneo em T2/T5, D no maximo 2")
408 lines.append("")
409
410 lines.append("## Resumo quantitativo")
411 rows = [(name, stats, sol) for name, _ok, sol, stats in runs]
412 lines.append(metrics_table(rows))
413 lines.append("")
414
415 lines.append("## Primeiros passos do Backtracking")
416 lines.append("Passo | Atribuicao | Estado parcial | Acao")
417 lines.append("----|---|---|---")
418 for i, row in enumerate(first_n_rows(trace_bt, 12), start=1):
419     lines.append(f"{i} | {row['var']}={row['value']} | {row['state']} | {row['status']}")
420 lines.append("")
421
422 lines.append("## Primeiros passos do Forward Checking")
423 lines.append("Passo | Atribuicao | Estado parcial | Acao")
424 lines.append("----|---|---|---")
425 for i, row in enumerate(first_n_rows(trace_fc, 12), start=1):
426     lines.append(f"{i} | {row['var']}={row['value']} | {row['state']} | {row['status']}")
427 lines.append("")
428
429 lines.append("## Primeiros passos do Backjumping")
430 lines.append("Passo | Atribuicao | Estado parcial | Acao")
431 lines.append("----|---|---|---")
432 for i, row in enumerate(first_n_rows(trace_bj, 12), start=1):
433     lines.append(f"{i} | {row['var']}={row['value']} | {row['state']} | {row['status']}")
434 lines.append("")
435
436 return "\n".join(lines).rstrip() + "\n"
437
438
439 def main() -> None:
440     runs: list[tuple[str, bool, dict[str, str], Stats]] = []
441
442     ok_bt, sol_bt, st_bt = solve_backtracking_like("BT",
443         select_fixed, use_fc=False, use_lcv=False)
444     ok_mrv, sol_mrv, st_mrv = solve_backtracking_like("BT+MRV",
445         select_mrv, use_fc=False, use_lcv=True)
446     ok_deg, sol_deg, st_deg = solve_backtracking_like("BT+MRV+
447         Degree", select_mrv_degree, use_fc=False, use_lcv=True)

```



```

445     ok_fc, sol_fc, st_fc = solve_backtracking_like("Forward
         Checking", select_mrv_degree, use_fc=True, use_lcv=True)
446     ok_bj, sol_bj, st_bj = solve_backjumping()
447
448     runs.append(("BT", ok_bt, sol_bt, st_bt))
449     runs.append(("BT+MRV", ok_mrv, sol_mrv, st_mrv))
450     runs.append(("BT+MRV+Degree", ok_deg, sol_deg, st_deg))
451     runs.append(("FC", ok_fc, sol_fc, st_fc))
452     runs.append(("BJ", ok_bj, sol_bj, st_bj))
453
454     report = build_report(runs, st_bt.trace, st_fc.trace, st_bj.
         trace)
455     out_dir = Path(__file__).resolve().parent.parent / "results"
456     out_dir.mkdir(parents=True, exist_ok=True)
457     out_file = out_dir / "q4_resultados.md"
458     out_file.write_text(report, encoding="utf-8")
459
460     print("Q4 concluida.")
461     print(f"Arquivo gerado: {out_file}")
462     for name, ok, sol, stats in runs:
463         sol_txt = "(" + ",".join(sol.get(t, "-") for t in TURNOS) +
            ")" if sol else "-"
464         print(
465             f"{name}: ok={ok} sol={sol_txt} "
466             f"estados={stats.states_explored} backtracks={stats.
                backtracks} "
467             f"prof_max={stats.max_depth} jumps={stats.jumps}"
468         )
469
470
471 if __name__ == "__main__":
472     main()

```

A.5 Questao 5

```

1  """Questao 5 - Minimax e poda Alpha-Beta.
2
3  Implementacao autoral:
4  - Minimax completo
5  - Alpha-Beta (ordem original)
6  - Alpha-Beta (ordem reordenada para maximizar podas)
7  - Minimax limitado (profundidade 2 com heuristica)
8  - Experimento com alteracao de 3 folhas
9  """
10
11 from __future__ import annotations
12
13 from dataclasses import dataclass, field
14 from math import inf
15 from pathlib import Path

```

```

16
17
18 ORIGINAL = [3, 5, 6, 9, 1, 2, 0, -1, 7, 4, 5, 6]
19 CHILDREN = ["B", "C", "D"] # filhos de A
20
21 # Reordenacao para aumentar podas:
22 # - visitar D antes (forca alpha alto cedo)
23 # - em cada no MIN, visitar menor folha primeiro
24 ROOT_REORDER = ["D", "B", "C"]
25 LEAF_ORDER_REORDER = {
26     "B": [0, 1, 2, 3],          # [3,5,6,9] (ja crescente)
27     "C": [3, 2, 0, 1],          # [-1,0,1,2]
28     "D": [1, 2, 3, 0],          # [4,5,6,7]
29 }
30
31 LIMIT_HEURISTICS = [4, 7, 2, 5, 6, 1]
32 MODIFIED = [8, 8, 8, 9, 1, 2, 0, -1, 7, 4, 5, 6]
33
34
35 def to_groups(leaves: list[int]) -> dict[str, list[int]]:
36     return {
37         "B": leaves[0:4],
38         "C": leaves[4:8],
39         "D": leaves[8:12],
40     }
41
42
43 @dataclass
44 class SearchStats:
45     internal_nodes: int = 0
46     leaves_visited: int = 0
47     leaves_pruned: int = 0
48     expansion_order: list[str] = field(default_factory=list)
49     alpha_beta_log: list[dict] = field(default_factory=list)
50
51     @property
52     def total_nodes_visited(self) -> int:
53         return self.internal_nodes + self.leaves_visited
54
55
56 def minimax(leaves: list[int]) -> tuple[int, dict[str, int],
57     SearchStats]:
58     stats = SearchStats()
59     groups = to_groups(leaves)
60
61     # Visita raiz A (MAX)
62     stats.internal_nodes += 1
63     values: dict[str, int] = {}
64
65     for child in CHILDREN:
66         stats.expansion_order.append(child)

```

```

66         stats.internal_nodes += 1 # no MIN
67         child_leaves = groups[child]
68         child_min = inf
69         for v in child_leaves:
70             stats.leaves_visited += 1
71             child_min = min(child_min, v)
72         values[child] = int(child_min)
73
74     return max(values.values()), values, stats
75
76
77 def alpha_beta(
78     leaves: list[int],
79     root_order: list[str] | None = None,
80     leaf_reorder: dict[str, list[int]] | None = None,
81 ) -> tuple[int, dict[str, int], SearchStats]:
82     stats = SearchStats()
83     groups = to_groups(leaves)
84     root_order = root_order or CHILDREN
85     leaf_reorder = leaf_reorder or {}
86
87     alpha = -inf
88     beta_root = inf
89     values: dict[str, int] = {}
90     step = 1
91
92     stats.internal_nodes += 1 # raiz A
93     for child in root_order:
94         stats.expansion_order.append(child)
95         stats.internal_nodes += 1 # no MIN filho
96         beta = beta_root
97         min_value = inf
98         child_leaves = groups[child]
99         idx_order = leaf_reorder.get(child, [0, 1, 2, 3])
100
101         for pos, idx in enumerate(idx_order):
102             v = child_leaves[idx]
103             stats.leaves_visited += 1
104             min_value = min(min_value, v)
105             beta = min(beta, min_value)
106             stats.alpha_beta_log.append(
107                 {
108                     "step": step,
109                     "node": child,
110                     "leaf_value": v,
111                     "alpha": alpha,
112                     "beta": beta,
113                     "cut": beta <= alpha,
114                 }
115             )
116             step += 1

```

```

117         if beta <= alpha:
118             # folhas restantes deste no MIN podadas
119             remaining = len(idx_order) - (pos + 1)
120             stats.leaves_pruned += remaining
121             break
122
123     values[child] = int(min_value)
124     alpha = max(alpha, min_value)
125
126     return int(alpha), values, stats
127
128
129
130 def minimax_limited_depth2(heuristics: list[int]) -> tuple[int,
131 list[int]]:
132     """Raiz MAX; 3 filhos MIN; cada MIN com 2 avaliacoes
133     heurísticas."""
134     mins = [
135         min(heuristics[0], heuristics[1]),
136         min(heuristics[2], heuristics[3]),
137         min(heuristics[4], heuristics[5]),
138     ]
139     return max(mins), mins
140
141
142 def build_report() -> str:
143     lines = []
144     lines.append("# Q5 - Resultados Minimax e Alpha-Beta")
145     lines.append("")
146
147     v_mm, mm_children, mm_stats = minimax(ORIGINAL)
148     lines.append("## 1) Minimax (ordem original)")
149     lines.append(f"Valor na raiz A: {v_mm}")
150     lines.append(f"Filhos MIN: B={mm_children['B']}, C={mm_children['C']}, D={mm_children['D']}")
151     lines.append(f"Nos visitados: {mm_stats.total_nodes_visited} (internos={mm_stats.internal_nodes}, folhas={mm_stats.leaves_visited})")
152     lines.append("")
153
154     v_ab, ab_children, ab_stats = alpha_beta(ORIGINAL)
155     lines.append("## 2) Alpha-Beta (ordem original)")
156     lines.append(f"Valor na raiz A: {v_ab}")
157     lines.append(f"Filhos MIN: B={ab_children['B']}, C={ab_children['C']}, D={ab_children['D']}")
158     lines.append(f"Nos visitados: {ab_stats.total_nodes_visited} "
159                 f"(internos={ab_stats.internal_nodes}, folhas={ab_stats.leaves_visited}); "
160                 f"folhas podadas={ab_stats.leaves_pruned}")

```

```

161 lines.append("")
162 lines.append("Passo | No MIN | Folha avaliada | alpha | beta |
    Poda?")
163 lines.append("----|---|----|----|----|---")
164 for row in ab_stats.alpha_beta_log:
165     cut = "sim" if row["cut"] else "nao"
166     lines.append(
167         f"{row['step']} | {row['node']} | {row['leaf_value']} |
            "
168         f"{row['alpha']} | {row['beta']} | {cut}"
169     )
170 lines.append("")
171
172 v_ab_re, ab_re_children, ab_re_stats = alpha_beta(
173     ORIGINAL,
174     root_order=ROOT_REORDER,
175     leaf_reorder=LEAF_ORDER_REORDER,
176 )
177 lines.append("## 3) Alpha-Beta reordenado (maximizar podas)")
178 lines.append(f"Valor na raiz A: {v_ab_re}")
179 lines.append(f"Ordem de filhos na raiz: {ROOT_REORDER}")
180 lines.append(f"Filhos MIN: B={ab_re_children['B']}, C={
    ab_re_children['C']}, D={ab_re_children['D']}")
181 lines.append(
182     f"Nos visitados: {ab_re_stats.total_nodes_visited} "
183     f"(internos={ab_re_stats.internal_nodes}, folhas={
        ab_re_stats.leaves_visited}); "
184     f"folhas podadas={ab_re_stats.leaves_pruned}"
185 )
186 lines.append("")
187
188 v_lim, mins_lim = minimax_limited_depth2(LIMIT_HEURISTICS)
189 lines.append("## 4) Minimax limitado (profundidade 2)")
190 lines.append(f"Heurísticas de corte: {LIMIT_HEURISTICS}")
191 lines.append(f"Valores MIN por bloco: {mins_lim}")
192 lines.append(f"Valor na raiz: {v_lim}")
193 lines.append("")
194
195 v_mm_mod, mm_mod_children, mm_mod_stats = minimax(MODIFIED)
196 v_ab_mod, ab_mod_children, ab_mod_stats = alpha_beta(MODIFIED)
197 lines.append("## 5) Experimento: alterar 3 folhas")
198 lines.append(f"Folhas alteradas: [3,5,6] -> [8,8,8]")
199 lines.append(
200     f"Minimax modificado: raiz={v_mm_mod}, "
201     f"B={mm_mod_children['B']}, C={mm_mod_children['C']}, D={
        mm_mod_children['D']}, "
202     f"nos={mm_mod_stats.total_nodes_visited}"
203 )
204 lines.append(
205     f"Alpha-Beta modificado: raiz={v_ab_mod}, "

```

```

206         f"B={ab_mod_children['B']}, C={ab_mod_children['C']}, D={
            ab_mod_children['D']}, "
207         f"nos={ab_mod_stats.total_nodes_visited}, folhas podadas={
            ab_mod_stats.leaves_pruned}"
208     )
209     lines.append("")
210
211     lines.append("## 6) Comparacao Minimax x Alpha-Beta")
212     lines.append("Cenario | Minimax nos | Alpha-Beta nos | Folhas
        podadas AB")
213     lines.append("---|---:|---:|---:")
214     lines.append(f"Original | {mm_stats.total_nodes_visited} | {
        ab_stats.total_nodes_visited} | {ab_stats.leaves_pruned}")
215     lines.append(f"Reordenado (AB) | - | {ab_re_stats.
        total_nodes_visited} | {ab_re_stats.leaves_pruned}")
216     lines.append(f"Modificado | {mm_mod_stats.total_nodes_visited}
        | {ab_mod_stats.total_nodes_visited} | {ab_mod_stats.
        leaves_pruned}")
217     lines.append("")
218
219     return "\n".join(lines).rstrip() + "\n"
220
221
222 def main() -> None:
223     out_dir = Path(__file__).resolve().parent.parent / "results"
224     out_dir.mkdir(parents=True, exist_ok=True)
225     out_file = out_dir / "q5_resultados.md"
226     out_file.write_text(build_report(), encoding="utf-8")
227
228     v_mm, mm_children, mm_stats = minimax(ORIGINAL)
229     v_ab, ab_children, ab_stats = alpha_beta(ORIGINAL)
230     v_ab_re, _ab_re_children, ab_re_stats = alpha_beta(
231         ORIGINAL,
232         root_order=ROOT_REORDER,
233         leaf_reorder=LEAF_ORDER_REORDER,
234     )
235     v_mm_mod, mm_mod_children, _mm_mod_stats = minimax(MODIFIED)
236
237     print("Q5 concluida.")
238     print(f"Arquivo gerado: {out_file}")
239     print(
240         "Minimax original:",
241         f"raiz={v_mm}",
242         f"B={mm_children['B']}",
243         f"C={mm_children['C']}",
244         f"D={mm_children['D']}",
245         f"nos={mm_stats.total_nodes_visited}",
246     )
247     print(
248         "Alpha-Beta original:",
249         f"raiz={v_ab}",

```

```

250         f"B={ab_children['B']}",
251         f"C={ab_children['C']}",
252         f"D={ab_children['D']}",
253         f"nos={ab_stats.total_nodes_visited}",
254         f"podas_folhas={ab_stats.leaves_pruned}",
255     )
256     print(
257         "Alpha-Beta reordenado:",
258         f"raiz={v_ab_re}",
259         f"nos={ab_re_stats.total_nodes_visited}",
260         f"podas_folhas={ab_re_stats.leaves_pruned}",
261     )
262     print(
263         "Minimax modificado:",
264         f"raiz={v_mm_mod}",
265         f"B={mm_mod_children['B']}",
266         f"C={mm_mod_children['C']}",
267         f"D={mm_mod_children['D']}",
268     )
269
270
271 if __name__ == "__main__":
272     main()

```

A.6 Questao 6

```

1  """Questao 6 - Monte Carlo Tree Search (MCTS) simplificado.
2
3  Modelo simplificado da decisao na raiz (colunas C1..C4), sem
4  biblioteca externa.
5  Inclui:
6  - 10 iteracoes detalhadas (selecao/expansao/rollout/retropropagacao)
7  - Tabela UCT
8  - Sensibilidade ao parametro C
9  - Comparacao rollout aleatorio vs semi-guloso
10 """
11 from __future__ import annotations
12
13 from dataclasses import dataclass, field
14 from math import log, sqrt
15 from pathlib import Path
16 from random import Random
17
18
19 COLUMNS = ["C1", "C2", "C3", "C4"]
20 TRUE_WIN_PROB = {"C1": 0.64, "C2": 0.58, "C3": 0.44, "C4": 0.34}
21
22

```

```

23 @dataclass
24 class Node:
25     move: str | None
26     parent: "Node | None" = None
27     children: dict[str, "Node"] = field(default_factory=dict)
28     visits: int = 0
29     wins: int = 0
30
31     def uct(self, exploration: float) -> float:
32         if self.parent is None:
33             return float("inf")
34         if self.visits == 0:
35             return float("inf")
36         return (self.wins / self.visits) + exploration * sqrt(log(
37             max(1, self.parent.visits)) / self.visits)
38
39 def expand_one(node: Node) -> Node:
40     for mv in COLUMNS:
41         if mv not in node.children:
42             child = Node(move=mv, parent=node)
43             node.children[mv] = child
44             return child
45     return node
46
47
48 def select_uct(node: Node, exploration: float) -> Node:
49     unvisited = [ch for ch in node.children.values() if ch.visits
50                 == 0]
51     if unvisited:
52         return unvisited[0]
53     return max(node.children.values(), key=lambda ch: ch.uct(
54         exploration))
55
56 def rollout_random(rng: Random, move: str) -> int:
57     # Politica totalmente aleatoria (maior variancia)
58     p = TRUE_WIN_PROB[move]
59     return 1 if rng.random() < p else 0
60
61 def rollout_semi_greedy(rng: Random, move: str) -> int:
62     # Politica semi-gulosa: ganho medio por priorizar centro/
63     # bloqueio
64     bonus = {"C1": 0.12, "C2": 0.10, "C3": 0.03, "C4": 0.02}
65     p = min(0.95, TRUE_WIN_PROB[move] + bonus[move])
66     return 1 if rng.random() < p else 0
67
68 def backpropagate(node: Node, result: int) -> None:
69     cur = node

```



```

70     while cur is not None:
71         cur.visits += 1
72         cur.wins += result
73         cur = cur.parent
74
75
76 def run_mcts(
77     iterations: int,
78     exploration: float,
79     rollout_policy: str,
80     seed: int,
81     record_trace: bool = False,
82 ) -> tuple[Node, list[dict]]:
83     rng = Random(seed)
84     root = Node(move=None)
85     trace: list[dict] = []
86
87     for it in range(1, iterations + 1):
88         expanded = False
89         if len(root.children) < len(COLUMNS):
90             chosen = expand_one(root)
91             expanded = True
92         else:
93             chosen = select_uct(root, exploration)
94
95         if rollout_policy == "semi":
96             result = rollout_semi_greedy(rng, chosen.move or "C1")
97         else:
98             result = rollout_random(rng, chosen.move or "C1")
99
100         backpropagate(chosen, result)
101
102         if record_trace:
103             trace.append(
104                 {
105                     "iter": it,
106                     "path": f"raiz -> {chosen.move}",
107                     "expanded": chosen.move if expanded else f"(
108                         reuso) {chosen.move}",
109                     "rollout": "vitoria" if result == 1 else "
110                         derrota",
111                     "n": chosen.visits,
112                     "w": chosen.wins,
113                 }
114             )
115
116     return root, trace
117
118 def uct_table(root: Node, exploration: float) -> list[dict]:
119     rows = []

```

```

119     for mv in COLUMNS:
120         ch = root.children.get(mv)
121         if ch is None:
122             rows.append({"move": mv, "n": 0, "w": 0, "uct": float("inf")})
123             continue
124             rows.append({"move": mv, "n": ch.visits, "w": ch.wins, "uct": ch.uct(exploration)})
125     return rows
126
127
128 def best_move(root: Node) -> str:
129     if not root.children:
130         return COLUMNS[0]
131     return max(root.children.values(), key=lambda ch: (ch.visits, ch.wins)).move or COLUMNS[0]
132
133
134 def diversity_score(root: Node) -> int:
135     return sum(1 for mv in COLUMNS if mv in root.children and root.children[mv].visits > 0)
136
137
138 def normalized_entropy(root: Node) -> float:
139     total = sum(ch.visits for ch in root.children.values())
140     if total == 0:
141         return 0.0
142     probs = [ch.visits / total for ch in root.children.values() if ch.visits > 0]
143     # Entropia normalizada por log2(4)=2
144     import math
145
146     h = -sum(p * math.log(p, 2) for p in probs)
147     return h / 2.0
148
149
150 def stability_experiment(
151     exploration: float,
152     rollout_policy: str,
153     runs: int = 40,
154     iterations: int = 40,
155     base_seed: int = 9000,
156 ) -> dict:
157     best_counts = {mv: 0 for mv in COLUMNS}
158     diversity_vals = []
159     avg_winrate_vals = []
160
161     for i in range(runs):
162         root, _ = run_mcts(
163             iterations=iterations,
164             exploration=exploration,

```

```

165         rollout_policy=rollout_policy,
166         seed=base_seed + i,
167         record_trace=False,
168     )
169     bm = best_move(root)
170     best_counts[bm] += 1
171     diversity_vals.append(normalized_entropy(root))
172
173     total_n = sum(ch.visits for ch in root.children.values())
174     total_w = sum(ch.wins for ch in root.children.values())
175     avg_winrate_vals.append((total_w / total_n) if total_n > 0
176                             else 0.0)
176
177     stable_move = max(best_counts, key=lambda mv: best_counts[mv])
178     stability = best_counts[stable_move] / runs
179     return {
180         "best_counts": best_counts,
181         "stable_move": stable_move,
182         "stability": stability,
183         "avg_diversity": sum(diversity_vals) / len(diversity_vals),
184         "avg_winrate": sum(avg_winrate_vals) / len(avg_winrate_vals)
185     },
186 }
187
188 def build_report() -> str:
189     lines = []
190     lines.append("# Q6 - Resultados MCTS")
191     lines.append("")
192
193     # 10 iteracoes detalhadas com C=1.4 e rollout semi-guloso
194     root10, trace10 = run_mcts(
195         iterations=10,
196         exploration=1.4,
197         rollout_policy="semi",
198         seed=707,
199         record_trace=True,
200     )
201     lines.append("## 1) MCTS detalhado (10 iteracoes, C=1.4,
202                 rollout semi-guloso)")
203     lines.append("Iter | Caminho selecionado | No expandido/reusado
204                 | Resultado rollout | Atualizacao no escolhido (N,W)")
205     lines.append("----:|---|---|---|---")
206     for row in trace10:
207         lines.append(
208             f"{row['iter']} | {row['path']} | {row['expanded']} | {
209                 row['rollout']} | ({row['n']},{row['w']})"
210         )
211     lines.append("")
212
213     lines.append("## 2) Tabela UCT apos 10 iteracoes")

```

```

211 lines.append("Jogada | Visitas N | Vitorias W | UCT")
212 lines.append("---|---:|---:|---:")
213 for row in uct_table(root10, 1.4):
214     lines.append(f"{row['move']} | {row['n']} | {row['w']} | {
        row['uct']:.4f}")
215 lines.append(f"Melhor jogada por visitas: **{best_move(root10)
        }**")
216 lines.append("")
217
218 # Sensibilidade de C
219 lines.append("## 3) Sensibilidade ao parametro C (rollout semi-
        guloso)")
220 lines.append("C | Movimento mais estavel | Estabilidade |
        Diversidade media | Taxa media de vitoria")
221 lines.append("---|---|---:|---:|---:")
222 c_results = []
223 for c in [0.1, 1.4, 3.0]:
224     res = stability_experiment(c, rollout_policy="semi")
225     c_results.append((c, res))
226     lines.append(
227         f"{c} | {res['stable_move']} | {100.0*res['stability
            ']:.1f}% | "
228         f"{res['avg_diversity']:.2f} | {res['avg_winrate']:.3f}
            "
229     )
230 lines.append("")
231
232 # Rollout random vs semi
233 lines.append("## 4) Rollout aleatorio vs semi-guloso (C=1.4)")
234 rnd = stability_experiment(1.4, rollout_policy="random")
235 semi = stability_experiment(1.4, rollout_policy="semi")
236 lines.append("Politica | Movimento mais estavel | Estabilidade
        | Diversidade media | Taxa media de vitoria")
237 lines.append("---|---|---:|---:|---:")
238 lines.append(
239     f"Aleatorio | {rnd['stable_move']} | {100.0*rnd['stability
        ']:.1f}% | "
240     f"{rnd['avg_diversity']:.2f} | {rnd['avg_winrate']:.3f}"
241 )
242 lines.append(
243     f"Semi-guloso | {semi['stable_move']} | {100.0*semi['
        stability']:.1f}% | "
244     f"{semi['avg_diversity']:.2f} | {semi['avg_winrate']:.3f}"
245 )
246 lines.append("")
247
248 # Efeito do numero de iteracoes
249 lines.append("## 5) Efeito do numero de iteracoes (C=1.4)")
250 lines.append("Iteracoes | Rollout | Movimento mais estavel |
        Estabilidade | Diversidade media | Taxa media de vitoria")
251 lines.append("---|---|---|---:|---:|---:")

```

```

252     for it in [10, 20, 40, 80, 120]:
253         semi_it = stability_experiment(
254             exploration=1.4,
255             rollout_policy="semi",
256             runs=40,
257             iterations=it,
258             base_seed=9000,
259         )
260         lines.append(
261             f"{it} | Semi-guloso | {semi_it['stable_move']} | "
262             f"{100.0*semi_it['stability']:.1f}% | "
263             f"{semi_it['avg_diversity']:.2f} | {semi_it['"
264             f"avg_winrate']:.3f}"
265         )
266     for it in [10, 20, 40, 80, 120]:
267         rnd_it = stability_experiment(
268             exploration=1.4,
269             rollout_policy="random",
270             runs=40,
271             iterations=it,
272             base_seed=9000,
273         )
274         lines.append(
275             f"{it} | Aleatorio | {rnd_it['stable_move']} | {100.0*
276             rnd_it['stability']:.1f}% | "
277             f"{rnd_it['avg_diversity']:.2f} | {rnd_it['avg_winrate
278             ']:.3f}"
279         )
280     lines.append("")
281
282     return "\n".join(lines).rstrip() + "\n"
283
284 def main() -> None:
285     out_dir = Path(__file__).resolve().parent.parent / "results"
286     out_dir.mkdir(parents=True, exist_ok=True)
287     out_file = out_dir / "q6_resultados.md"
288     out_file.write_text(build_report(), encoding="utf-8")
289
290     root10, _ = run_mcts(
291         iterations=10,
292         exploration=1.4,
293         rollout_policy="semi",
294         seed=707,
295         record_trace=False,
296     )
297     print("Q6 concluida.")
298     print(f"Arquivo gerado: {out_file}")
299     print("Melhor jogada apos 10 iteracoes (C=1.4, semi):",
300           best_move(root10))
301     for row in uct_table(root10, 1.4):

```

```
298         print(  
299             f"{row['move']}: N={row['n']} W={row['w']} UCT={row['  
300                 uct']: .4f}"  
301         )  
302  
303 if __name__ == "__main__":  
304     main()
```